

Beyond Worst-Case: Regret-Optimal Supervisory Control of Partially-Known Discrete-Event Systems

Jianing Zhao, Bohan Cui, Huayang Cui, Sadegh Soudjani, Dimos V. Dimarogonas, Rupak Majumdar, Xiang Yin

Abstract—This paper investigates regret-optimal supervisory control for reachability tasks in partially-known discrete-event systems (DES). We consider systems whose transition structure is fixed but only partially known to the supervisor, where the feasibility of certain events can be revealed through online exploration. To quantify the performance loss caused by such structural uncertainty, we adopt regret as the optimality criterion, defined relative to the optimal supervisor that could be synthesized if the actual plant were known in advance. We formalize the partially-known DES framework and construct a knowledge-based tripartite transition system that captures supervisory decisions, uncontrollable event occurrences, and the revelation of unknown transition structures. Based on this representation, we develop an exact Pareto value-iteration algorithm that propagates realization-wise cost profiles and synthesizes a reachability-guaranteeing supervisor with minimum worst-case regret. Our results demonstrate that regret serves as a meaningful performance measure for supervisory control in partially-known DES, and our method is both correct and effective in practice.

Index Terms—Discrete-event systems, supervisory control, regret minimization.

I. INTRODUCTION

A. Motivation

Discrete-Event Systems (DES) are an important class of systems with discrete state spaces and event-triggered dynamics [4]. They play an essential role in modeling, analysis and control of high-level behaviors for engineering cyber-physical systems such as manufacturing systems, embedded software, and communication networks. In the context of DES, one of the central problems is how to enforce the closed-loop properties of the system. Supervisory control theory, initiated by Ramadge and Wonham [26], provides a powerful formal framework, where a feedback supervisor is synthesized to dynamically enable/disable controllable events such that the controlled system satisfies some desired specifications, such as safety, liveness, and nonblockingness [29], [35], [36].

In addition to enforcing qualitative specifications, one important problem in the supervisory control theory is to synthesize supervisors optimally in terms of some quantitative performance measures, which is referred to as the *optimal supervisory control problem* and has drawn considerable attention; see, e.g., [1], [8], [13], [20], [30]. Early studies on optimal

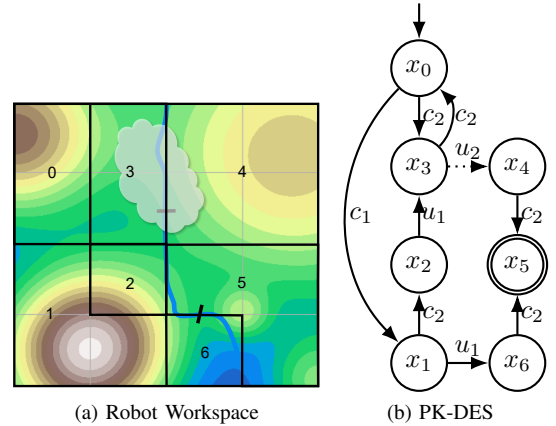


Fig. 1. A motivating example, where a robot needs to reach region 6 from region 0 with partially-known environment information.

supervisory control considered event and control costs, as well as accumulated costs associated with reaching marked states [30]. This line of research has subsequently been extended to various performance criteria and control settings, including near-optimal and online control [9], optimal control under partial observation [13], [21], and more recent formulations based on alternative task-oriented performance measures [19]. These works provide systematic approaches for synthesizing supervisors that simultaneously satisfy the prescribed specifications and optimize quantitative system performance.

In most of existing works in the supervisory control theory, uncertainties are modeled as uncontrollable events. These models, while suitable for describing disturbances, fall short in capturing the scenarios where the system structure is inherently unknown and the supervisor is required to actively explore the system to uncover the actual configuration. Relying solely on the worst-case analysis may lead to a highly conservative supervisor. To be more specific, let us consider a simple motivating example where a robot moves in a partially-known workspace as shown in Fig 1a, the corresponding transition model is as shown in Fig 1b. We assume that the robot knows the terrain and the bridge between regions 5 and 6, but is uncertain about the bridge between regions 3 and 4 due to a cloud. The goal is to reach region 5 from region 0 with minimal energy cost. However, traversing mountainous terrain incurs significantly higher energy consumption compared to the plains. A worst-case strategy follows path $0 \rightarrow 1 \rightarrow 6 \rightarrow 5$ to prevent unnecessary backtracking. However, this may lead to a heavy regret if a bridge does exist at region 3, as the robot

J. Zhao, S. Soudjani and R. Majumdar are with the Max-Planck Institute for Software Systems, Kaiserslautern 67663, Germany. E-mail: {jzhao, sadegh, rupak}@mpi-sws.org.

B. Cui, H. Cui and X. Yin are with the School of Automation and Intelligent Sensing, Shanghai Jiao Tong University, Shanghai 200240, China. E-mail: {bohan_cui, chybje1023, yinxian}@sjtu.edu.cn.

D. V. Dimarogonas is with the School of Electrical Engineering and Computer Science, KTH Royal Institute of Technology, Stockholm SE 10044, Sweden. E-mail: dimos@kth.se.

would have missed a shortcut. A more adaptive approach is to first check region 3. If a bridge is present, the robot takes the shortcut and saves energy; otherwise, it backtracks to 0. While this strategy may have a slightly higher worst-case cost, it renders much less regret and takes the potential advantage of exploring the unknown regions.

The above example illustrates that even in a fully-controllable setting, worst-case performance may not always be suitable for evaluating a planning strategy. A systematic approach is needed to quantitatively balance the trade-off between exploration and conservatism. More fundamentally, such scenarios involve uncertainty about the actual transition structure of the system, which is not directly captured by standard DES models. This motivates us to introduce a new class of DES with partially-known structures and to study optimal supervisory control under such structural uncertainty.

B. Our Results and Contributions

Motivated by the above example, in this paper, we address a new type of optimal supervisory control problem for DES whose transition structures are *partially known*. Specifically, we introduce a new modeling framework in a new partially-known setting. This type of uncertainty is fundamentally different from standard nondeterminism [10] and partial observation [6]. In a nondeterministic DES, multiple transition outcomes are incorporated into the plant dynamics, whereas in a PK-DES the actual transition structure is fixed but initially unknown. In a partially observed DES, the plant structure is known but the supervisor may have incomplete information about the current state or executed events. In contrast, a partially-known DES assumes that the current state is observable while the feasibility of some transitions is initially unknown. Therefore, PK-DESs are designed to capture persistent but learnable structural uncertainty that can be resolved through online exploration.

To evaluate the performance of a supervisor under such structural uncertainty, we adopt the notion of *regret*, which measures the performance loss relative to the optimal supervisor that could have been designed if the actual plant realization were known in advance. This criterion provides a natural way to balance the potential benefit of exploration against the cost incurred when exploration is unsuccessful. To isolate the decision-making issues arising from partial knowledge, we focus on the reachability objective [28], which allows us to clearly reveal how exploration and acquired structural information affect supervisory decisions. Our objective is to synthesize a supervisor that guarantees the reachability property for every compatible plant while minimizing the worst-case regret. The main contributions of this paper are summarized as follows.

- First, we establish a new DES modeling framework for structural information uncertainty and exploration. In particular, the proposed partially-known DES (PK-DES) explicitly captures the evolution of knowledge as unknown transition structures are revealed during system execution, thereby providing a formal basis for supervisory decision making with online exploration. Based on this framework, we formulate the regret-optimal supervisory control problem under reachability requirements.

- Second, we develop an exact dynamic-programming approach for synthesizing a regret-optimal supervisor. The main challenge lies in the coupling of two different sources of uncertainty: the uncertainty in event occurrences induced by uncontrollable events and the structural information uncertainty arising from initially unknown transitions. To capture their interaction, we construct a tripartite transition system that explicitly separates supervisory decisions, event occurrences, and information revelation. Leveraging the structural properties of this system, we develop a novel Pareto value-iteration algorithm to solve the regret-optimal supervisory control problem. This avoids explicit enumeration of supervisory strategies while maintaining exact regret optimality.
- Finally, we establish the theoretical guarantees of the proposed approach, including finite termination, completeness, and regret optimality of the synthesized supervisor. A robotic case study further demonstrates that the proposed regret-optimal strategy can exploit beneficial unknown transitions while limiting the cost of unsuccessful exploration, thereby reducing the conservatism of conventional worst-case supervisory control.

C. Related Works

We discuss related works in the literature from the following three perspectives.

Supervisory Control of Nondeterministic DES. Supervisory control has also been extensively studied for nondeterministic DES, where the plant may exhibit multiple possible transition outcomes. Early works established supervisory-control frameworks for such systems [10], while more recent studies have mainly adopted simulation- and bisimulation-based formulations to preserve the branching behaviors of nondeterministic plants. Representative results include similarity-enforcing control [15], [32], bisimilarity-enforcing control [31], and their extensions to partial observation, nonblockingness, and maximal permissiveness [16], [17], [33]. These works assume a known nondeterministic transition structure and synthesize supervisors against all possible branching behaviors. In contrast, a PK-DES has a fixed but initially unknown transition structure that is progressively revealed through exploration. Therefore, supervisory decisions depend on accumulated structural knowledge, which is not captured by standard nondeterministic DES models.

Supervisory Control of Partially-Observed DES. Supervisory control under partial observation has been extensively studied when the supervisor cannot fully distinguish system behaviors. Classical approaches characterize the feasibility of control through observability-related conditions and synthesize supervisors based on state estimates induced by observation histories [6], [37]. More recent studies have developed compact game-based synthesis structures and considered different forms of unobservable behavior [22], [27], as well as modular and networked supervisory control under partial observation [14], [34]. Optimal supervisory control under partial observation has also been investigated for quantitative objectives [13], [21]. These works generally assume that the plant structure is

known, while uncertainty arises from incomplete observations of its execution. In contrast, a PK-DES assumes observable states but initially unknown transition feasibility that can be progressively learned through exploration.

Regret-Optimal Reactive Synthesis. Regret minimization has been studied in quantitative graph games as an alternative to conventional worst-case synthesis [5]. Early work [7] considered regret minimization for weighted reachability games using graph unfolding. Subsequent studies developed regret-based reactive synthesis under different classes of environment strategies, including unrestricted and positional strategies, and for different quantitative objectives [3], [11], [12]. Regret has also been employed in robotic reactive synthesis to encourage cooperation while preserving task guarantees [23], [24]. More closely related to the present work, our previous study [39] developed regret-based temporal-logic planning in partially-known environments using a knowledge-based game and an efficient value-iteration algorithm. This regret-based exploration principle has also been extended to heterogeneous multi-robot task allocation [18]. However, none of the above works considers the simultaneous presence of uncertainty in event occurrences induced by uncontrollable events and information uncertainty arising from initially unknown transition structures. These two sources of uncertainty have fundamentally different semantics: uncontrollable event occurrences may vary during system execution, whereas the unknown transition structure is fixed in the actual system and is progressively revealed through exploration. Their interaction therefore naturally leads to a three-player game. From a robotic planning perspective, the same setting arises when a robot operates in a partially-known environment while its own motion or action outcomes are also nondeterministic.

A preliminary version of the present work appeared in [38]. The present paper substantially extends the conference version by developing an exact Pareto value-iteration approach that avoids explicit strategy enumeration and establishes complete correctness and optimality guarantees. Moreover, the proposed framework subsumes our previous regret-optimal planning formulation [39]: when all events are controllable, the event-occurrence player becomes trivial, and, after the standard product construction with the task automaton, the resulting game reduces to the two-player partially-known planning setting.

D. Organization

The remaining part of this paper is organized as follows. In Section II, we review optimal supervisory control for fully-known DES under reachability requirements. In Section III, we introduce the partially-known DES framework and formulate the regret-optimal supervisory control problem. In Section IV, we construct the knowledge-based tripartite transition system and reformulate the problem as a three-player graph game. In Section V, we develop the Pareto value-iteration algorithm for regret-optimal supervisor synthesis and establish its theoretical guarantees. In Section VI, we present a robotic case study to demonstrate the proposed approach. Finally, Section VII concludes the paper.

II. SUPERVISORY CONTROL OF FULLY-KNOWN DES

A. System Model

Let Σ be a finite set of events. A *string* is a finite sequence of events and we denote by Σ^* the set of all strings over Σ including the empty string ϵ . For integer n , we denote by Σ^n the set of strings with length n . For string $s \in \Sigma^*$, the length of s is denoted by $|s|$ with $|\epsilon| = 0$. A language $L \subseteq \Sigma^*$ is a set of strings. The prefix-closure of L is denoted by $\bar{L}$, i.e., $\bar{L} = \{u \in \Sigma^* : \exists v \in \Sigma^* \text{ s.t. } uv \in L\}$.

We consider a *fully-known* discrete-event system modeled by a deterministic finite-state automaton (DFA)

$$G = (X, \Sigma, \delta_G, x_0, X_m)$$

where X is the finite set of states, Σ is the finite set of events, $\delta_G : X \times \Sigma \rightarrow X$ is the partial transition function, $x_0 \in X$ is the initial state, and $X_m \subseteq X$ is the set of marked states. The transition function δ_G can also be extended to $\delta_G : X \times \Sigma^* \rightarrow X$ in the usual manner. The language generated by G from state x is defined by $\mathcal{L}(G, x) = \{s \in \Sigma^* : \delta_G(x, s)!\}$, where “!” means “is defined”. We also define $\mathcal{L}(G, Q) := \bigcup_{x \in Q} \mathcal{L}(G, x)$ as the language generated from a set of states $Q \subseteq X$, define the language generated by G is $\mathcal{L}(G) := \mathcal{L}(G, x_0)$, and the language marked by G is $\mathcal{L}_m(G) = \{s \in \mathcal{L}(G) : \delta_G(s) \in X_m\}$. For any $s \in \mathcal{L}(G)$, we write $\delta_G(x_0, s)$ simply as $\delta_G(s)$. For any $x \in X$, we define $\Lambda_G(x) = \{\sigma \in \Sigma : \delta_G(x, \sigma)!\}$ as the set of active events at x . For any $s \in \mathcal{L}(G)$, we also write $\Lambda_G(\delta_G(s))$ as $\Lambda_G(s)$ for simplicity.

In the supervisory control framework, a supervisor can restrict the behavior of the system G by dynamically disabling/enabling some system events. In this setting, the event set Σ is partitioned as $\Sigma = \Sigma_c \dot{\cup} \Sigma_{uc}$, where Σ_c is the set of controllable events and Σ_{uc} is the set of uncontrollable events. A control decision $\gamma \in 2^\Sigma$ is said to be *admissible* if $\Sigma_{uc} \subseteq \gamma$, namely, uncontrollable events can never be disabled. Particularly, in this paper, we consider the control decisions in which *at most one controllable event* is included. This setting is without loss of generality since it does not influence the controllability of the system. Formally, we define

$$\Gamma = \{\gamma \in 2^\Sigma : \Sigma_{uc} \subseteq \gamma \wedge |\gamma \setminus \Sigma_{uc}| \leq 1\}$$

as the set of admissible control decisions. Then, a supervisor is a mapping

$$S : \mathcal{L}(G) \rightarrow \Gamma \quad (1)$$

We denote by $\Psi(G)$ the set of all supervisors in G . Moreover, we use the notation S/G to represent the controlled system and the language generated by S/G , denoted by $\mathcal{L}(S/G)$, is defined recursively in the following manners:

- i) $\epsilon \in \mathcal{L}(S/G)$; and
- ii) for any $s \in \Sigma^*$, $\sigma \in \Sigma$ we have $s\sigma \in \mathcal{L}(S/G)$ iff $s\sigma \in \mathcal{L}(G)$, $s \in \mathcal{L}(S/G)$, and $\sigma \in S(s)$.

The language marked by S/G is denoted by $\mathcal{L}_m(S/G) = \mathcal{L}(S/G) \cap \mathcal{L}_m(G)$.

In this paper, we aim to synthesis a supervisor enforcing *reachability objective*, which requires that the controlled DES will eventually visit the marked states, i.e.,

$$\begin{aligned} \forall s \in \mathcal{L}(S/G), \exists n \in \mathbb{N}, \forall t \in \mathcal{L}(S/G)/s : \\ |t| \geq n \Rightarrow \overline{\{st\}} \cap \mathcal{L}_m(G) \neq \emptyset. \end{aligned} \quad (2)$$

Accordingly, we define

$$\Psi_W(G) = \{S \in \Psi(G) : S/G \text{ satisfies (2)}\}$$

as the set of all winning supervisors for the reachability.

In this paper, we consider the optimal supervisory control in a quantitative manner. To this end, for each event, we define the cost function $w : \Sigma \rightarrow \mathbb{R}_{\geq 0}$ describing the cost incurred whenever the DES generates an event. Given a finite string $t = \sigma_1 \cdots \sigma_n \in \mathcal{L}(G)$, we define $\text{cost}(t) = \sum_{i=1}^n w(\sigma_i)$ as the total cost along the string s . For a string $s \in \mathcal{L}_m(G)$, we define its cost as the total cost when it first hits the set of marked states X_m , i.e.,

$$\begin{aligned} \text{cost}(s) &= \text{cost}(t), \\ t &\in \overline{\{s\}} \cap \mathcal{L}_m(G) \wedge t' \notin \mathcal{L}_m(G), \forall t' \in \overline{\{t\}} \setminus \{t\}. \end{aligned} \quad (3)$$

Given a winning supervisor $S \in \Psi_W(G)$, we define the cost of the controlled system S/G as

$$\text{cost}(S/G) = \max_{s \in \mathcal{L}_m(S/G)} \text{cost}(s). \quad (4)$$

B. Worst-Case-based Supervisory Control

In the literature on quantitative supervisory control of DES for reachability tasks, the performance of a supervisor is commonly evaluated in terms of the worst-case accumulated cost over all possible behaviors of the controlled system; see for example, [21], [25], [30]. Such a worst-case optimization problem can essentially be viewed as a two-player zero-sum min-max reachability game, where the supervisor selects an admissible control decision to minimize the accumulated cost, while the plant resolves the occurrence of enabled events in the worst-case manner. Formally, the optimality problem is to synthesize a supervisor $S^* \in \Psi_W(G)$ such that

$$\text{cost}(S^*/G) \leq \text{cost}(S/G), \forall S \in \Psi_W(G). \quad (5)$$

To solve such an optimal supervisory control problem, it essentially suffices to solve a min-max reachability problem. To this end, we represent the interaction between the supervisor and the plant by the following bipartite transition system (BTS) [37], which explicitly captures the alternating choices of the supervisor's control decisions and the plant's event occurrences.

Definition 1 (Bipartite Transition System). *Given system G , we construct its bipartite transition system (BTS) as*

$$\mathcal{B} = (Q_X, Q_Y, x_0, h_{XY}, h_{YX}, \Sigma, \Gamma)$$

where

- $Q_X = X \cup \{d\}$ is the set of X -states, where d is the deadlocked state;
- $Q_Y = X \times \Gamma$ is the set of Y -states;
- $x_0 \in Q_Y$ is the initial X -state;
- $h_{XY} : Q_X \times \Gamma \rightarrow Q_Y$ is the transition function from X -states to Y -states define by: for any $x \in Q_X$, $\gamma \in \Gamma$ and $y = (x, \gamma) \in Q_Y$, $y = h_{XY}(x, \gamma)$;
- $h_{YX} : Q_Y \times (\Sigma \cup \{\epsilon\}) \rightarrow Q_X$ is the transition function from Y -states to X -states define by: for any $y = (x, \gamma) \in Q_Y$, we have

– if $\Lambda_G(x) \cap \gamma \neq \emptyset$, we define

$$h_{YX}(y, \sigma) = \delta(x, \sigma), \forall \sigma \in \Lambda_G(x) \cap \gamma \quad (6)$$

– if $\Lambda_G(x) \cap \gamma = \emptyset$, we define

$$h_{YX}(y, \epsilon) = d \quad (7)$$

- Σ is the set of events in G ;
- Γ is the set of admissible control decisions in G .

Here we note that the marked states in G are implicitly included in X -states of $\mathcal{B}$, i.e., $X_m \subset Q_X$. For the sake of writing convenience, in this paper, we use $\text{Succ}(\cdot)$ to denote the successor-states of a given state in the graph, i.e.,

- for each $x \in Q_X$, we have

$$\text{Succ}(x) = \{h_{XY}(x, \gamma) : \gamma \in \Gamma\}$$

- for each $x \in Q_Y$, we have

$$\text{Succ}(x) = \{h_{YX}(y, \sigma) : \sigma \in \Sigma \cup \{\epsilon\} \text{ s.t. } h_{YX}(y, \sigma) \neq d\}$$

Hereafter, the detailed definition of $\text{Succ}(\cdot)$ for other kinds of graphs will be omitted.

To capture the optimality, we define a cost function for $\mathcal{B}$ as follows: for each $x \in Q_X \setminus \{d\}$ and each $y \in h_{XY}(x)$, we define $w_{\mathcal{B}}(x, y) = 0$; for each $y \in Q_Y$ and each $x' = h_{YX}(y, \sigma) \in h_{YX}(y)$, we define $w_{\mathcal{B}}(y, x') = w(\sigma)$.

Then, the optimal supervisor can be obtained by solving a two-player *min-max reachability game* on the BTS, as summarized in Algorithm 1; see, e.g., [2] for min-cost reachability games on weighted graphs. We note that some existing works formulate the worst-case optimal supervisory control problem directly in terms of controlled languages, optimal submachines, or dynamic programming, rather than explicitly using a two-player game representation [21], [30]. Nevertheless, these formulations can be equivalently interpreted through the same underlying min-max structure: at each step, the supervisor selects an admissible control decision to minimize the worst-case accumulated cost, while the subsequent event occurrence is resolved among all events enabled by the selected control decision.

Given the optimal strategy π^* returned by Algorithm 1, in execution, the optimal supervisor S^* can work as follows: at each instant, S^* remembers the current X -state x and pick the decision γ such that $h_{XY}(x, \gamma) = \pi^*(x)$. Then we update the current to Y -state according to γ and wait for the next event to occur. After that, we update the current state again to X -state and so forth.

Although uncertainties in the system can be effectively captured by uncontrollable events under the above framework, however, as illustrated by the motivating example in the Introduction, there also exist uncertainties arising from initially unknown transition structures rather than from uncontrollable behaviors. In particular, once the system visits the predecessor state of such an unknown transition, whether the transition exists becomes known to the supervisor and remains known thereafter. In what follows, we start to introduce a new modeling framework of *partially-known DES* to explicitly capture this type of information uncertainty.

Algorithm 1: Min-Max Game

Input: A BTS $\mathcal{B}$ with the set of marked states X_m and a cost function $w_{\mathcal{B}}$

Output: Optimal strategy $\pi^*: Q_X \rightarrow Q_Y$ and the optimal minmax value minmax

```

1 foreach  $x \in Q_X$  do
2   if  $x \in X_m$  then
3      $\text{Val}^{(0)}(x) \leftarrow 0$  and  $\pi^{(0)}(x) \leftarrow \Lambda_G(x) \cup \Sigma_{uc}$ 
4   else
5      $\text{Val}^{(0)}(x) \leftarrow \infty$ 
6 repeat
7   foreach  $y \in Q_Y$  do
8      $\text{Val}^{(k+1)}(y) \leftarrow \max_{x' \in \text{Succ}(y)} (\text{Val}^{(k)}(x') + w_{\mathcal{B}}(y, x'))$ 
9   foreach  $x \in Q_X$  do
10     $\text{Val}^{(k+1)}(x) \leftarrow \min_{y \in \text{Succ}(x)} (\text{Val}^{(k)}(y) + w_{\mathcal{B}}(x, y))$ 
11     $\pi^{(k+1)}(x) \leftarrow \arg \min_{y \in \text{Succ}(x)} (\text{Val}^{(k)}(y) + w_{\mathcal{B}}(x, y))$ 
12   $k \leftarrow k + 1$ 
13 until  $\forall z \in Q_X \cup Q_Y : \text{Val}^{(k+1)}(z) = \text{Val}^{(k)}(z)$ ;
14 return  $\pi^{(k)}, \text{Val}^{(k)}(x_0)$ 

```

III. SUPERVISORY CONTROL IN PARTIALLY-KNOWN DES

In this section, we develop a modeling framework for *partially-known DES* to capture information uncertainty arising from transition structures that are initially unknown but become revealed through system execution. More specifically, we consider a partially-known DES characterized by the following assumptions:

- A1 The supervisor has complete knowledge of the state space and the possible state transitions of the system;
- A2 At each state, the set of activated events is fixed in the actual system, but the supervisor may not know *a priori* which events are actually activated;
- A3 Once the system execution visits a state, the supervisor observes the set of activated events at that state precisely, and this information remains known thereafter.

A. Modeling of Partially-Known DES

Formally, we capture the *information uncertainty* by the following partially-known DES.

Definition 2 (Partially-Known DES). A *partially-known DES* (PK-DES) is a 6-tuple

$$\mathbb{G} = (X, \Sigma, \delta, \Delta, x_0, X_m)$$

where X is the set of states; Σ is the set of events; $\delta : X \times \Sigma \rightarrow X$ is the transition function; x_0 is the initial state; X_m is the set of marked states; different from the DFA,

$$\Delta : X \rightarrow 2^{2^\Sigma}$$

is the event-pattern function that assigns each state a family of activated events.

The intuition of the PK-DES $\mathbb{G}$ is explained as follows. Essentially, PK-DES is used to describe the *possible world* for the system. That is, the system has some prior information regarding the possible events of each unknown state but does not know which ones are activated before the system trajectory actually visits it. Therefore, in the PK-DES $\mathbb{G}$, for each state $x \in X$, we have

$$\Delta(x) = \{o_1, o_2, \dots, o_{|\Delta(x)|}\}$$

where each $o_i \in 2^\Sigma$ is called an *event-pattern* representing a possible set of activated events at state x . For convenience, we refer each $o_i \in \Delta(x)$ to as an *observation* at state x , since the system “observes” the activated events when visiting state x . Therefore, for each $x \in X$, we say x is a *known state* if $|\Delta(x)| = 1$; and *unknown state* if $|\Delta(x)| > 1$. Accordingly, we partition the state space as $X = X_k \cup X_{uk}$ where X_k is the set of known states and X_{uk} is the set of unknown states. We refer the visit to an unknown state to as an *exploration*.

Definition 3 (Compatible DES). Given $\mathbb{G}$, a DFA $G = (X, \Sigma, \delta_G, x_0, X_m)$ is a *compatible DES* with $\mathbb{G}$, denoted by $G \in \mathbb{G}$, if for any $x \in X$, we have i) $\Lambda_G(x) \in \Delta(x)$; and ii) for each $\sigma \in \Lambda_G(x)$, we have $\delta_G(x, \sigma) = \delta(x, \sigma)$.

Example 1 (Running Example). We use the motivating example in Figure 1 as the running example to illustrate the proposed framework. The corresponding PK-DES is shown in Figure 1b, where $\Sigma_c = \{c_1, c_2\}$, $\Sigma_{uc} = \{u_1, u_2\}$ and $X_m = \{x_5\}$. The event-pattern function Δ is given by

$$\begin{aligned} \Delta(x_0) &= \{\{c_1, c_2\}\}, & \Delta(x_1) &= \{\{c_2, u_1\}\}, \\ \Delta(x_2) &= \{\{u_1\}\}, & \Delta(x_3) &= \{\{c_2\}, \{c_2, u_2\}\}, \\ \Delta(x_4) &= \{\{c_2\}\}, & \Delta(x_6) &= \{\{c_2\}\}. \end{aligned}$$

Therefore, x_3 is the unique unknown state, while all other non-marked states are known. The two event patterns $\{c_2\}$ and $\{c_2, u_2\}$ at x_3 represent, respectively, the situations where the bridge from region 3 to region 4 is unavailable or available. The actual event pattern at x_3 is fixed throughout the system execution. Accordingly, there are two compatible DESs, denoted by G_1 and G_2 , which differ only at x_3 , i.e., $\Lambda_{G_1}(x_3) = \{c_2\}$ and $\Lambda_{G_2}(x_3) = \{c_2, u_2\}$.

B. Knowledge Update by Exploration

In the partially-known setting, a supervisor cannot be synthesized only based on the finite string generated by the system. In addition, the observed successor-pattern at each state should also be taken into consideration. To be specific, if the system trajectory visits a known state, then there would not be any useful information about the partially-known DES, since $\Delta(x)$ is already a singleton. Only when the system trajectory visits unknown state, it will gain new information and successor-pattern at this state will become known from then on.

To capture the result of an exploration, we call a tuple $\kappa = (x(\kappa), o(\kappa)) \in X \times 2^\Sigma$ as a *knowledge state*, where $o \in \Delta(x)$ is the set of activated events of x that becomes known to the agent after exploring x . We denote by $\text{Kw} = \{\kappa \in X \times 2^\Sigma :$

$o(\kappa) \in \Delta(x(\kappa))$ the set of all possible knowledge states. A *history* in $\mathbb{G}$ is a finite sequence

$$\bar{h} = (x_0, o_0)\sigma_0(x_1, o_1)\sigma_1 \cdots (x_n, o_n) \in (\text{Kw} \cdot \Sigma)^* \text{Kw}$$

such that i) for any $i = 0, 1, \dots, n-1$, we have $\sigma_i \in o_i$ and $x_{i+1} = \delta(x_i, \sigma_i)$; ii) for any $i, j = 1, \dots, n$, we have $x_i = x_j \Rightarrow o_i = o_j$. For such history $\bar{h}$, we call $\sigma_1\sigma_2 \cdots \sigma_{n-1} \in X^*$ its string. We denote by $\mathcal{H}(\mathbb{G})$ and $\mathcal{L}(\mathbb{G})$ the set of all finite histories and paths in PK-DES $\mathbb{G}$, respectively.

Given knowledge on the PK-DES will be accumulated along a history, which is captured by the following concept of *knowledge set*. A knowledge set $\mathcal{K} = \{\kappa_1, \dots, \kappa_{|\mathcal{K}|}\}$ where $\kappa_i \in \mathcal{K}$ is a set of knowledge states such that

$$\forall \kappa, \kappa' \in \mathcal{K} : x(\kappa) = x(\kappa') \Rightarrow o(\kappa) = o(\kappa').$$

We denote by $\mathbb{KW}$ the set of all knowledge sets. With a slight abuse of notation, for each $x \in X$, we write $x \in \mathcal{K}$ if $(x, o) \in \mathcal{K}$ for some observation $o \in \Delta(x)$. We denote by $o_{\mathcal{K}}(x) \in \Delta(x)$ the unique observation such that $(x, o_{\mathcal{K}}(x)) \in \mathcal{K}$.

During the system execution, the system maintains a knowledge set to record its exploration history. Once a new unknown state is explored, the knowledge set is updated according to the following function. Given a knowledge set $\mathcal{K} \in \mathbb{KW}$ and a knowledge state $\kappa \in \text{Kw}$, we have

$$\text{update}(\mathcal{K}, \kappa) = \begin{cases} \mathcal{K}, & \text{if } x(\kappa) \in \mathcal{K} \\ \mathcal{K} \cup \{\kappa\}, & \text{otherwise} \end{cases} \quad (8)$$

Example 2 (Running Example Cont.). *We still consider the PK-DES in Figure 1b. Initially, the knowledge set is $\mathcal{K}_0 = \emptyset$. When the system visits the unknown state x_3 , it observes either $\{c_2\}$ or $\{c_2, u_2\}$ and hence acquires the knowledge state*

$$\kappa_1 = (x_3, \{c_2\}) \quad \text{or} \quad \kappa_2 = (x_3, \{c_2, u_2\}),$$

respectively. Accordingly, the knowledge set is updated to

$$\mathcal{K}_1 = \text{update}(\mathcal{K}_0, \kappa_1) = \{\kappa_1\}$$

or

$$\mathcal{K}_2 = \text{update}(\mathcal{K}_0, \kappa_2) = \{\kappa_2\}.$$

Since x_3 is the only unknown state, $\mathcal{K}_1$ and $\mathcal{K}_2$ uniquely identify the compatible DESs G_1 and G_2 , respectively.

C. Strategic Supervisor

Under the setting of partially-known DES, it is no longer sufficient to synthesize a supervisor in (1), since the supervisor should decide the control decision based on both of what the system has visited and what it has known. Therefore, we define the notion of *strategic supervisor*.

Definition 4 (Strategic Supervisor). *A strategic supervisor is a function*

$$\mathbb{S} : \mathcal{H}(\mathbb{G}) \rightarrow \Gamma$$

such that for any $\bar{h} = \kappa_0\sigma_1\kappa_1\sigma_2 \cdots \kappa_n$ where $\kappa_i = (x_i, o_i)$, we have

$$\mathbb{S}(\bar{h}) = \{\sigma_c\} \cup \Sigma_{uc}, \quad \sigma_c \in o_n \cap \Sigma_c,$$

i.e., the supervisor makes a control decision based on the observed set of activated events at x_n .

For convenience, we denote by $\Phi(\mathbb{G})$ the set of all strategic supervisors in $\mathbb{G}$.

Given a strategic supervisor $\mathbb{S} \in \Phi(\mathbb{G})$ and an actual DES $G \in \mathbb{G}$, the controlled system, denoted by $\mathbb{S}/G$, whose language $\mathcal{L}(\mathbb{S}/G)$ is recursively defined as:

- i) $\epsilon \in \mathcal{L}(\mathbb{S}/G)$;
- ii) for any $\bar{h} = \kappa_0\sigma_1\kappa_1\sigma_2 \cdots \kappa_n$, we have $\sigma_1\sigma_2 \cdots \sigma_n \in \mathcal{L}(\mathbb{S}/G)$ iff $\sigma_1\sigma_2 \cdots \sigma_{n-1} \in \mathcal{L}(\mathbb{S}/G)$ and $\sigma_n \in \mathbb{S}(\bar{h})$.

In the partially-known setting, since the actual DES $G \in \mathbb{G}$ is unknown *a priori*, we aim to synthesize a strategic supervisor $\mathbb{S} \in \Phi(\mathbb{G})$ such that

$$\forall G \in \mathbb{G} : \mathcal{L}(\mathbb{S}/G) \text{ satisfies (2)} \quad (9)$$

We denote by $\Phi_W(\mathbb{G}) \subseteq \Phi(\mathbb{G})$ the set of all winning strategic supervisors for the reachability objective.

D. Problem Formulation

To evaluate the performance of a strategic supervisor $\mathbb{S}$, a natural approach is to still consider the *worst-case cost* similar to the definition of (4) among all possible actual DES, i.e.,

$$\text{Cost}_{\text{worst}}(\mathbb{S}) := \max_{G \in \mathbb{G}} \text{cost}(\mathbb{S}/G) \quad (10)$$

A supervisor minimizing this criterion can be synthesized by a straightforward extension of the min-max game in Algorithm 1. That is, in addition to resolving event occurrences in the worst-case manner, the unknown event patterns are also resolved adversarially whenever they are first revealed. Therefore, the resulting supervisor minimizes the accumulated cost against both the worst-case event occurrences and the worst-case realization of the partially-known plant.

However, this metric cannot capture the potential benefit obtained from exploring unknown states. To address this issue, we propose to use *regret* to evaluate the performance of a strategic supervisor, which is defined as follows.

Definition 5 (Regret). *Given a partially-known DES $\mathbb{G}$, the regret of a supervisor $\mathbb{S} \in \Phi(\mathbb{G})$ is*

$$\text{Reg}(\mathbb{S}) = \max_{G \in \mathbb{G}} \left(\text{cost}(\mathbb{S}/G) - \min_{\mathbb{S}' \in \Phi_W(G)} \text{cost}(\mathbb{S}'/G) \right)$$

The intuition is explained as follows. For each strategic supervisor $\mathbb{S} \in \Phi(\mathbb{G})$ and each possible actual DES $G \in \mathbb{G}$, $\text{cost}(\mathbb{S}/G)$ is the actual cost incurred when applying $\mathbb{S}$ to this specific DES, while $\min_{\mathbb{S}' \in \Phi_W(G)} \text{cost}(\mathbb{S}'/G)$ is the *best-response cost* if G had been known to the system with hindsight. Therefore, their difference is the regret of applying $\mathbb{S}$ in G . Since the system does not know which $G \in \mathbb{G}$ the actual DES is, the regret of $\mathbb{S}$ is defined as the worst-case regret among all $G \in \mathbb{G}$.

Finally, we formally formulate the problem that we solve in this paper as follows.

Problem 1 (Regret-Optimal Supervisory Control). *Given a partially-known DES $\mathbb{G}$, synthesize a strategic supervisor $\mathbb{S} \in \Phi_W(\mathbb{G})$ such that*

$$\forall \mathbb{S}' \in \Phi_W(\mathbb{G}) : \text{Reg}(\mathbb{S}) \leq \text{Reg}(\mathbb{S}'). \quad (11)$$

Example 3 (Running Example Cont.). *Still, we consider the running example in Figure 1b, where $w(c_2) = 2$, $w(c_1) = w(u_1) = 10$ and $w(u_2) = 1$. Recall that there are two compatible DESs $G_1, G_2 \in \mathbb{G}$, where $\Lambda_{G_1}(x_3) = \{c_2\}$ and $\Lambda_{G_2}(x_3) = \{c_2, u_2\}$. That is, the bridge from region 3 to region 4 is unavailable in G_1 and available in G_2 . Let $\mathbb{S}_1$ be the strategic supervisor that does not explore x_3 and directly takes the route through x_1 , and let $\mathbb{S}_2$ be the strategic supervisor that first explores x_3 . If the bridge is available, $\mathbb{S}_2$ takes the shortcut through x_4 ; otherwise, it returns to x_0 and follows the same route as $\mathbb{S}_1$. Then,*

- for G_1 , we have $\text{cost}(\mathbb{S}_1/G_1) = 22$, $\text{cost}(\mathbb{S}_2/G_1) = 26$, and $\min_{S \in \Psi_W(G_1)} \text{cost}(S/G_1) = 22$;
- for G_2 , we have $\text{cost}(\mathbb{S}_1/G_2) = 22$, $\text{cost}(\mathbb{S}_2/G_2) = 5$, and $\min_{S \in \Psi_W(G_2)} \text{cost}(S/G_2) = 5$.

Therefore, $\text{Reg}(\mathbb{S}_1) = \max\{22 - 22, 22 - 5\} = 17$, whereas $\text{Reg}(\mathbb{S}_2) = \max\{26 - 22, 5 - 5\} = 4$.

Remark 1. *The partially-known DES framework we proposed above captures a type of uncertainty that is fundamentally different from those considered in standard supervisory control. First, uncontrollable events model the lack of control over event occurrences under a fully-known plant structure, whereas the partially-known DES considered here captures information uncertainty regarding the plant structure itself. In particular, the actual set of activated events at an unknown state is fixed in the underlying system but is not known to the supervisor a priori; such information is revealed once the state is explored and remains known thereafter. The proposed framework is also different from, and complementary to, the widely studied partially-observed setting in DES. Under partial observation, the plant model is typically known, while the supervisor may be uncertain about the current system state due to the occurrence of unobservable events. In contrast, in our setting, the system state is observable, but some structural information of the plant is initially unknown. The actual plant realization remains fixed throughout the execution, and the supervisor progressively refines its knowledge of this realization through exploration. Therefore, the partially-known DES framework provides a natural model for systems whose dynamics are fixed but only partially known before execution and can be gradually identified during operation.*

Essentially, the partially-known DES considered in this paper involves two types of uncertainty: the nondeterminism in event occurrences induced by uncontrollable events, and the information uncertainty arising from the initially unknown plant structure. To facilitate the analysis and synthesis of the regret-optimal supervisor, we transform the problem into an equivalent three-player graph game, where the X -player represents the supervisor, the Y -player captures the event-occurrence nondeterminism, and the Z -player captures the information uncertainty.

IV. KNOWLEDGE-BASED GAMES

In this section, we provide a game-theoretic formulation of the regret-optimal supervisory control problem. We first construct a knowledge-based tripartite transition system (TTS)

to explicitly represent the control decisions, event occurrences, and information revelation. We then introduce the notions of plays and player strategies on the TTS, which provide the basis for formulating and solving the regret-optimal control problem as a three-player graph game.

A. Tripartite Transition System

We aim to incorporate the knowledge set into the state space of the game arena and explicitly split the movements of the control decision, the system execution, and the non-determinism of the environment, which leads to the following notion of *tripartite transition system*.

Definition 6 (Tripartite Transition System). *Given a PK-DES $\mathbb{G}$, its tripartite transition system (TTS) is a tuple*

$$\mathcal{G} = (V_X, V_Y, V_Z, v_0, f_{XZ}, f_{ZY}, f_{YX}, \Sigma, \Gamma)$$

where

- $V_X = X \times \mathbb{KW}$ is the set of X -states;
- $V_Y = X \times \mathbb{KW} \times \Gamma$ is the set of Y -states;
- $V_Z = X \times \mathbb{KW} \times X \cup \{d\}$ is the set of Z -states;
- $v_0 = (x_0, \mathcal{K}_0) \in Q_X$ is the initial state with $\mathcal{K}_0$ being the initial knowledge set;
- $f_{XY} : V_X \times \Gamma \rightarrow V_Y$ is the transition function from X -states to Y -states defined by: for any $v_x = (x, \mathcal{K}) \in V_X$, $\gamma \in \Gamma$, and $v_y = (x, \mathcal{K}, \gamma) \in V_Y$, we have

$$v_y = f_{XY}(v_x, \gamma) \quad (12)$$

- $f_{YZ} : V_Y \times (\Sigma \cup \{\epsilon\}) \rightarrow V_Z$ is the transition function from Y -states to Z -states defined by: for any $v_y = (x, \mathcal{K}, \gamma) \in V_Y$, we have

– if $\gamma \cap o_{\mathcal{K}}(x) \neq \emptyset$, we define

$$f_{YZ}(v_y, \sigma) = (x, \mathcal{K}, \delta(x, \sigma)), \quad \forall \sigma \in \gamma \cap o_{\mathcal{K}}(x) \quad (13)$$

– if $\gamma \cap o_{\mathcal{K}}(x) = \emptyset$, we define

$$f_{YZ}(v_y, \epsilon) = d \quad (14)$$

- $f_{ZX} : V_Z \times 2^\Sigma \rightarrow V_X$ is the transition function from Z -states to X -states defined by: for any $v_z = (x, \mathcal{K}, x') \in V_Z \setminus \{d\}$ and any $o \in \Delta(x')$, we define

$$v_x = f_{ZX}(v_z, o) = (x', \mathcal{K}') \quad (15)$$

with $\mathcal{K}' = \text{update}(\mathcal{K}, (x', o))$;

- Σ is the set of events in G ;
- Γ is the set of admissible control decisions in G .

The intuition of the TTS is explained as follows. The graph is tripartite with three types of states: the X -states from which the system chooses a feasible control decision; the Y -states from which the system is executed according the control decision chosen before; and the Z -states from which the actual event-pattern is decided in the possible world.

Example 4 (Running Example Cont.). *For the PK-DES shown in Figure 1b, the corresponding TTS $\mathcal{G}$ is depicted in Figure 2, where the knowledge sets $\mathcal{K}_0$, $\mathcal{K}_1$, and $\mathcal{K}_2$ are given in Example 2. For simplicity, we omit the Z -states associated*

with known states, as well as the control decisions that cannot satisfy the reachability objective.

In particular, since x_3 is the only unknown state, when the system moves from x_0 to x_3 , the Z -state $(x_0, \mathcal{K}_0, x_3)$ resolves the actual event pattern at x_3 . Accordingly, its two successors are $(x_3, \mathcal{K}_1)$ and $(x_3, \mathcal{K}_2)$, corresponding to the two possible observations $\{c_2\}$ and $\{c_2, u_2\}$, respectively.

Remark 2. Let $X_{uk} = \{x_1, \dots, x_n\}$ be the set of unknown states and let $m_i = |\Delta(x_i)|$ denote the number of possible event patterns at x_i and $m = \max_{x \in X_{uk}} |\Delta(x)|$. Since each unknown state is either unexplored or associated with one of its possible event patterns, the number of knowledge sets satisfies $|\mathbb{KW}| \leq (m+1)^n$. Therefore, by the construction of the TTS, we have $|V| = O((m+1)^n |X|(|\Gamma| + |\Sigma|))$. Hence, the TTS is polynomial in the size of the underlying DES for fixed n and m , while its worst-case size grows exponentially with the number of unknown states.

For convenience, we denote by $V = V_X \cup V_Y \cup V_Z$ the state space of $\mathcal{G}$. For any $v \in V \setminus \{d\}$, we denote by $\mathcal{K}(v)$ the knowledge set contained in v . By the construction of TTS and the knowledge update in (8), for any $v, v' \in V_X \cup V_Y \cup V_Z \setminus \{d\}$ such that $v' \in \text{Succ}(v)$, we have $\mathcal{K}(v) \subseteq \mathcal{K}(v')$. Therefore, the knowledge set evolves monotonically along every path in $\mathcal{G}$. This monotonic evolution leads to the following structural property of the TTS $\mathcal{G}$.

Proposition 1. Every cycle in the TTS $\mathcal{G}$ is contained within a single knowledge layer. Specifically, for any cycle $v_0 v_1 \dots v_n$ with $v_n = v_0$, we have $\mathcal{K}(v_0) = \mathcal{K}(v_1) = \dots = \mathcal{K}(v_n)$.

Proof. By the monotonic evolution of the knowledge set, we have $\mathcal{K}(v_0) \subseteq \mathcal{K}(v_1) \subseteq \dots \subseteq \mathcal{K}(v_n) = \mathcal{K}(v_0)$. Hence, all inclusions must be equalities. $\square$

B. Plays and Strategies

We denote by $V = V_X \cup V_Y \cup V_Z$ the state space of $\mathcal{G}$. Furthermore, we define the set of marked states in $\mathcal{G}$ as $V_m = \{(x, \mathcal{K}) \in V_X : x \in X_m\}$. Given $\mathcal{G}$, we call a finite sequence

$$\rho = v_x^0 v_y^0 v_z^0 v_x^1 v_y^1 v_z^1 \dots v_x^n \in (V_X \cdot V_Y \cdot V_Z)^* V_X$$

a *play* if $v_x^0 = v_0$ and there are $\gamma_i \in \Gamma$, $\sigma_i \in \Sigma \cup \{\epsilon\}$ and $o_i \in 2^\Sigma$ such that $v_y^i = f_{XY}(v_x^i, \gamma_i)$, $v_z^i = f_{YZ}(v_y^i, \sigma_i)$, and $v_x^{i+1} = f_{ZX}(v_z^i, o_i)$ hold for each $i=0, 1, \dots, n-1$ and $v_x^i = v_x^n$ for $i = n$. It is obvious that each play in $\mathcal{G}$ induces a history in $\mathbb{G}$, i.e.,

$$\hbar_\rho = (x_0, o_0) \sigma_0 (x_1, o_1) \sigma_1 \dots (x_n, o_n) \in \mathcal{H}(\mathbb{G}).$$

For convenience, we still use ρ to denote the partial play that does not necessarily end with a X -state.

Since only edges from V_z to V_x represent actual movements, we define a weight function for $\mathcal{G}$ as $w_{\mathcal{G}} : V \times V \rightarrow \mathbb{R}_{\geq 0}$ where for any $v_z = (x, \mathcal{K}, x')$ and $v_x = (x', \mathcal{K}')$, we have $w_{\mathcal{G}}(v_z, v_x) = w(x, x')$, $w_{\mathcal{G}}(v_x, v_y) = w_{\mathcal{G}}(v_y, v_z) = 0$. The cost of a play $\rho = v_0 v_1 \dots v_n \in V^*$ is defined as

$$\text{cost}_{\mathcal{G}}(\rho) = \sum_{i=0}^{n-1} w_{\mathcal{G}}(v_i, v_{i+1}). \quad (16)$$

Given the above TTS $\mathcal{G}$, the strategies are functions

$$\pi_x : V^* V_X \rightarrow V_Y \cup \{\text{Null}\}, \quad \pi_y : V^* V_Y \rightarrow V_Z, \quad \pi_z : V^* V_Z \rightarrow V_X$$

for X -player, Y -player and Z -player, respectively. We denote by $\mathfrak{S}_X(\mathcal{G})$, $\mathfrak{S}_Y(\mathcal{G})$, $\mathfrak{S}_Z(\mathcal{G})$ the sets of all X -strategies, Y -strategies and Z -strategies, respectively. In particular, we say a strategy π is *positional* if $\forall \rho, \rho' : \text{last}(\rho) = \text{last}(\rho') \Rightarrow \pi(\rho) = \pi(\rho')$, where $\text{last}(\cdot)$ denotes the last state of a sequence. We denote by $\mathfrak{S}_j^1(\mathcal{G})$ the set of all positional strategies for j -player, where $j = X, Y, Z$. Given strategies $\pi_x \in \mathfrak{S}_X, \pi_y \in \mathfrak{S}_Y, \pi_z \in \mathfrak{S}_Z$, the outcome play $\rho_{\pi_x, \pi_y, \pi_z}$ is the unique sequence $v_0 v_1 \dots v_n \in V^* V_X$ such that

- $\forall i < n : v_i \in V_X \Rightarrow \pi_x(v_0 v_1 \dots v_i) = v_{i+1}$;
- $\forall i < n : v_i \in V_Y \Rightarrow \pi_y(v_0 v_1 \dots v_i) = v_{i+1}$;
- $\forall i < n : v_i \in V_Z \Rightarrow \pi_z(v_0 v_1 \dots v_i) = v_{i+1}$;
- $\pi_x(v_0 v_1 \dots v_n) = \text{Null}$.

Here we remark that, since the objective of the supervisor is to satisfy the reachability w.r.t. the set V_m , once the system trajectory visits V_m , it is unnecessary to restrict the behaviors of the system. To this end, in the TTS $\mathcal{G}$, for a play ρ satisfying $\text{last}(\rho) \in \text{Null}$, we set $\pi_x(\rho) = \text{Null}$.

Next, we aim to characterize the sets of strategies for the X, Y, Z -players that are sufficient to solve the problem.

For the Y -player, its behavior represents all possible occurrences of enabled events and is therefore not subject to any additional restriction. Hence, we let $\Pi_Y = \mathfrak{S}_Y(\mathcal{G})$ denote the set of all Y -strategies.

For the Z -player, it cannot play arbitrarily since the actual DES is fixed. Therefore, the Z -player must commit to a specific event-pattern it chooses at each unknown state when the game begins. To this end, we define the following notion of *strongly positional strategy*.

Definition 7 (Strongly Positional Strategies). We say a Z -strategy $\pi_z \in \mathfrak{S}_Z(\mathcal{G})$ is *strongly positional* if for any two plays $\rho, \rho' \in V^* V_Z$ where $\pi_z(\rho) = (x, \mathcal{K})$ and $\pi_z(\rho') = (x', \mathcal{K}')$, we have

- i) π_z is *positional*;
- ii) $x = x' \Rightarrow o_{\mathcal{K}}(x) = o_{\mathcal{K}'}(x')$.

We denote by $\Pi_Z \subseteq \mathfrak{S}_Z(\mathcal{G})$ the set of all strongly positional strategies for the Z -player.

Finally, we aim to find a winning X -strategy that ensures the outcome play visits the state in V_m . Therefore, we define

$$\Pi_X = \left\{ \pi_x \in \mathfrak{S}_X(\mathcal{G}) : \begin{array}{l} \text{last}(\rho_{\pi_x, \pi_y, \pi_z}) \in V_m, \\ \forall \pi_y \in \Pi_Y, \forall \pi_z \in \Pi_Z \end{array} \right\} \quad (17)$$

Now, similarly to Definition 5, we define the regret of an X -strategy $\pi_x \in \Pi_X$ in $\mathcal{G}$ as (18). It suffices to synthesize an X -strategy $\pi_X \in \Pi_X$ that minimizes $\text{Reg}_{\mathcal{G}}(\pi_x)$. In what follows, we present the solution by introducing two algorithms for micro and macro-strategy synthesis.

V. GAME-BASED STRATEGY SYNTHESIS

In this section, we present the algorithm of the regret-minimizing strategy synthesis on the knowledge-based game.

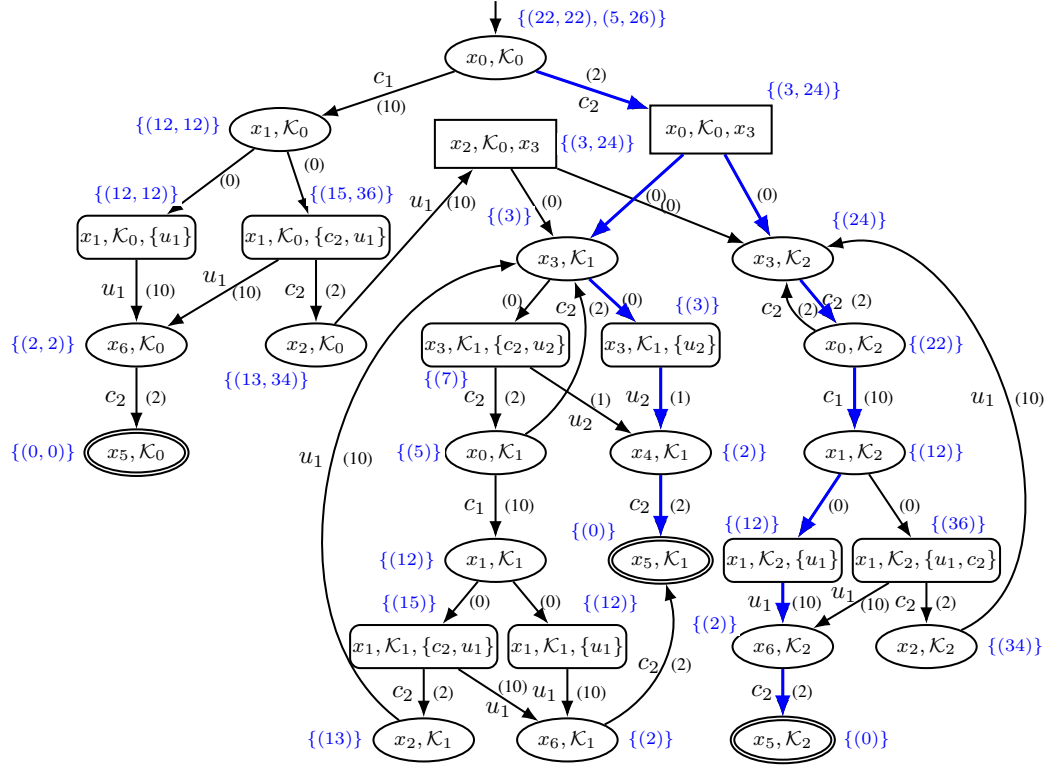


Fig. 2. Tripartite transition system of PK-DES in Fig 1b.

$$\text{Reg}_G(\pi_x) = \max_{\pi_z \in \Pi_Z} \left(\max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi_x, \pi_y, \pi_z}) - \min_{\pi'_x \in \Pi_X} \max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi'_x, \pi_y, \pi_z}) \right) \quad (18)$$

A. Regret-Minimizing Strategy Synthesis

Recall that X_{uk} is the set of unknown states in the PK-DES $\mathbb{G}$. We define

$$\Theta := \{\theta : X_{uk} \rightarrow 2^\Sigma \mid \theta(x) \in \Delta(x), \forall x \in X_{uk}\}. \quad (19)$$

as the set of all possible realizations of $\mathbb{G}$. That is, each realization $\theta \in \Theta$ assigns exactly one event pattern to each unknown state. Therefore, each realization $\theta \in \Theta$ uniquely induces a compatible DES, denoted by

$$G_\theta = (X, \Sigma, \delta_\theta, x_0, X_m),$$

such that

$$\Lambda_{G_\theta}(x) = \begin{cases} \theta(x), & x \in X_{uk}, \\ o, & x \in X_k \text{ and } \Delta(x) = \{o\}. \end{cases} \quad (20)$$

Given a knowledge set $\mathcal{K} \in \mathbb{KW}$, we denote by

$$\Theta_{\mathcal{K}} := \{\theta \in \Theta : \theta(x) = o_{\mathcal{K}}(x), \forall x \in X_{uk} \cap \mathcal{K}\} \quad (21)$$

the set of realizations consistent with the information contained in $\mathcal{K}$. In particular, $\Theta_{\mathcal{K}_0} = \Theta$ for the initial knowledge set. Each realization $\theta \in \Theta$ uniquely induces a strongly positional Z -strategy π_z^θ such that, for every $v_z = (x, \mathcal{K}, x') \in V_Z \setminus \{d\}$,

$$\pi_z^\theta(v_z) = f_{ZX}(v_z, \Lambda_{G_\theta}(x')). \quad (22)$$

Therefore, we have $\Pi_Z = \{\pi_z^\theta : \theta \in \Theta\}$. Moreover, $|\Theta_{\mathcal{K}}| = 1$ whenever the acquired knowledge uniquely identifies the actual DES.

Example 5 (Running Example Cont.). *For the running example, x_3 is the only unknown state and hence there are two possible realizations, $\Theta = \{\theta_1, \theta_2\}$, where $\theta_1(x_3) = \{c_2, u_2\}$ and $\theta_2(x_3) = \{c_2\}$. Thus, θ_1 corresponds to the realization in which the shortcut through event u_2 is available, whereas θ_2 corresponds to the realization in which it is unavailable. Recall that $\mathcal{K}_0 = \emptyset$, while $\mathcal{K}_1$ and $\mathcal{K}_2$ record the two possible observations at x_3 . Therefore, we have $\Theta_{\mathcal{K}_0} = \{\theta_1, \theta_2\}$, $\Theta_{\mathcal{K}_1} = \{\theta_1\}$ and $\Theta_{\mathcal{K}_2} = \{\theta_2\}$.*

For each realization $\theta \in \Theta$, the induced DES G_θ is fully known. Therefore, if the realization were known a priori, the supervisor could optimize its control decisions specifically for G_θ . We define the best response with respect to a realization as follows.

Definition 8 (Best Response). *Let $\theta \in \Theta$ be a realization and G_θ be the corresponding fully-known DES. The best response of the supervisor w.r.t. realization θ is defined as the minimum worst-case accumulated cost required to reach a marked state in G_θ , i.e.,*

$$\text{BR}(\theta) := \min_{S \in \Psi_W(G_\theta)} \text{cost}(S/G_\theta). \quad (23)$$

Since G_θ is fully known, its corresponding BTS $\mathcal{B}_\theta$ can be constructed, and $\text{BR}(\theta)$ can be computed directly by applying Algorithm 1. The best response $\text{BR}(\theta)$ serves as the full-information hindsight benchmark for realization θ in the subsequent regret evaluation.

According to the regret definition in (18), the regret under each realization is determined by comparing the actual worst-case cost incurred by the strategic supervisor with the corresponding best response. Since the best response $\text{BR}(\theta)$ can be computed beforehand for each realization θ , the remaining task is to characterize the actual costs of the strategic supervisor under the possible realizations.

Directly enumerating all possible runs and evaluating their costs is generally infeasible, especially since the TTS $\mathcal{G}$ may contain cycles and thus admit infinitely many runs. To avoid such explicit enumeration, we develop a Pareto value-iteration approach. The key idea is to propagate, at each state, the realization-wise worst-case cost information that is sufficient for the regret evaluation, while discarding redundant information through Pareto dominance. To formulate this value iteration, we first introduce several notions for representing, organizing, and updating such cost information.

The basic quantity propagated in the value iteration is the realization-wise cost-to-go from a TTS state. Since the realizations compatible with the current knowledge may lead to different worst-case costs, we collect these costs into a single realization-indexed object, referred to as a cost profile.

Definition 9 (Cost Profile). *Given a state $v \in V_X \cup V_Y \cup V_Z$ with knowledge set $\mathcal{K}$, for each realization $\theta \in \Theta_{\mathcal{K}}$, let $c_v^\theta \in \mathbb{R}_{\geq 0}$ denote the worst-case cost-to-go from v to a marked state under realization θ . We say a function*

$$\mathbf{c}_v : \Theta_{\mathcal{K}} \rightarrow \mathbb{R}_{\geq 0}$$

is called a cost profile of v if

$$\mathbf{c}_v(\theta) = c_v^\theta, \quad \forall \theta \in \Theta_{\mathcal{K}}.$$

Therefore, a cost profile collects the realization-wise worst-case cost-to-go values associated with the same state.

A single TTS state may admit different control choices and, consequently, different realization-wise cost tradeoffs. Therefore, a single cost profile is generally insufficient to represent all cost information that needs to be propagated during the value iteration. We thus associate each state with a collection of cost profiles.

Definition 10 (Cost Profile Set). *Given a state $v \in V_X \cup V_Y \cup V_Z$ with knowledge set $\mathcal{K}$, we denote by*

$$\mathcal{C}(v) = \{\mathbf{c}_v^1, \dots, \mathbf{c}_v^{m_v}\}$$

the cost profile set of v , where each $\mathbf{c}_v^i : \Theta_{\mathcal{K}} \rightarrow \mathbb{R}_{\geq 0}$ is a cost profile of v and $m_v = |\mathcal{C}(v)|$.

Since different cost profiles may represent different tradeoffs among the compatible realizations, not all profiles need to be retained. Particularly, a profile that incurs no smaller cost under any realization than another profile can be safely discarded. This motivates the following Pareto dominance relation.

Definition 11 (Pareto-Optimal Cost Profiles). *Let $\mathbf{c}$ and $\mathbf{c}'$ be two cost profiles defined on the same realization set $\Theta_{\mathcal{K}}$. We say that $\mathbf{c}$ weakly dominates $\mathbf{c}'$, denoted by $\mathbf{c} \preceq \mathbf{c}'$, if*

$$\mathbf{c}(\theta) \leq \mathbf{c}'(\theta), \quad \forall \theta \in \Theta_{\mathcal{K}}.$$

For a cost profile set $\mathcal{C}$, its Pareto-optimal subset is defined as

$$\text{Pareto}(\mathcal{C}) := \{\mathbf{c} \in \mathcal{C} \mid \nexists \mathbf{c}' \in \mathcal{C} \text{ such that } \mathbf{c}' \preceq \mathbf{c} \text{ and } \mathbf{c}' \neq \mathbf{c}\}.$$

In addition to comparing cost profiles, the value update at a Y -state requires combining the costs induced by different possible event occurrences. Since the event occurrence is resolved in the worst case, this combination must be performed separately for each realization. To capture this operation, given a nonempty finite set of cost profiles $\{\mathbf{c}_i\}_{i=1}^m$ defined on the same realization set $\Theta_{\mathcal{K}}$, i.e.,

$$\mathbf{c}_i : \Theta_{\mathcal{K}} \rightarrow \mathbb{R}_{\geq 0}, \quad i = 1, \dots, m,$$

we define their *componentwise maximum* as the cost profile

$$\text{CMax}(\{\mathbf{c}_i\}_{i=1}^m) : \Theta_{\mathcal{K}} \rightarrow \mathbb{R}_{\geq 0}$$

such that

$$\text{CMax}(\{\mathbf{c}_i\}_{i=1}^m)(\theta) := \max_{i \in \{1, \dots, m\}} \mathbf{c}_i(\theta), \quad \forall \theta \in \Theta_{\mathcal{K}}. \quad (24)$$

With the above notions in place, we are now ready to specify how cost-profile sets are propagated through the TTS during the value iteration. To this end, we introduce three value-iteration operators, which are formally defined in Definition 12. We explain the intuitions as follows.

- For the X -player, the supervisor selects an admissible control decision, represented by choosing one successor Y -state $v_y \in \text{Succ}(v_x)$. Each cost profile associated with such a successor therefore induces a candidate cost profile at v_x after accounting for the transition cost $w_{\mathcal{G}}(v_x, v_y)$. Since different control decisions may lead to different tradeoffs among the compatible realizations, these candidate profiles are generally not totally ordered. The operator T_X therefore collects the profiles induced by all admissible control decisions and retains their Pareto-optimal subset.
- For the Y -player, the control decision has already been fixed, while the actual event occurrence is resolved in the worst case. Hence, unlike the X -player, the update cannot select only one successor. Instead, for each possible event successor $v_z \in \text{Succ}(v_y)$, one cost profile is considered, and the worst-case cost is evaluated separately for every compatible realization. This is precisely captured by the componentwise maximum operator CMax . Since different combinations of successor profiles may still yield incomparable realization-wise costs, T_Y considers all such combinations and retains only the Pareto-optimal resulting profiles.
- For the Z -player, no additional worst-case choice is made. Instead, the transition from a Z -state resolves the event-pattern information at the successor state. For each realization $\theta \in \Theta_{\mathcal{K}}$, the fixed realization G_θ determines a unique successor X -state v_x^θ consistent with its event pattern. Consequently, the cost component corresponding to

Definition 12 (Value-Iteration Operators). Given a collection of cost-profile sets $\mathcal{C} = \{\mathcal{C}(v)\}_{v \in V}$, we define the value-iteration operators for the X -, Y -, and Z -players as follows.

- 1) For an X -state $v_x = (x, \mathcal{K}) \in V_X \setminus V_m$, we define

$$\mathsf{T}_X(\mathcal{C}, v_x) := \text{Pareto} \left(\left\{ w_{\mathcal{G}}(v_x, v_y) + \mathbf{c}_{v_y} \mid v_y \in \text{Succ}(v_x), \mathbf{c}_{v_y} \in \mathcal{C}(v_y) \right\} \right). \quad (25)$$

- 2) For a Y -state $v_y = (x, \mathcal{K}, \gamma) \in V_Y$, we define

$$\mathsf{T}_Y(\mathcal{C}, v_y) := \text{Pareto} \left(\left\{ \text{CMax} \left(\{w_{\mathcal{G}}(v_y, v_z) + \mathbf{c}_{v_z}\}_{v_z \in \text{Succ}(v_y)} \right) \mid \mathbf{c}_{v_z} \in \mathcal{C}(v_z), \forall v_z \in \text{Succ}(v_y) \right\} \right). \quad (26)$$

- 3) For a Z -state $v_z = (x, \mathcal{K}, x') \in V_Z \setminus \{d\}$ and each $\theta \in \Theta_{\mathcal{K}}$, by denoting $v_x^\theta := f_{ZX}(v_z, \Lambda_{G_\theta}(x'))$, we define

$$\mathsf{T}_Z(\mathcal{C}, v_z) := \text{Pareto} \left(\left\{ \mathbf{c}_{v_z} \mid \mathbf{c}_{v_x} \in \mathcal{C}(v_x), \forall v_x \in \text{Succ}(v_z), \mathbf{c}_{v_z}(\theta) = w_{\mathcal{G}}(v_z, v_x^\theta) + \mathbf{c}_{v_x^\theta}(\theta), \forall \theta \in \Theta_{\mathcal{K}} \right\} \right). \quad (27)$$

θ must be propagated through this realization-consistent successor. The operator T_Z therefore combines the corresponding components of the successor cost profiles into a new cost profile at v_z , and Pareto pruning is subsequently applied to remove dominated profiles.

For subsequent strategy synthesis, however, the retained cost profiles are insufficient, since one must also identify the successor state-profile choices that generate each retained profile. We therefore introduce the following notation to characterize such generating choices for the value-iteration operators.

For convenience, for each $i \in \{X, Y, Z\}$, let $\mathfrak{D}_i(\mathcal{C}, v_i)$ denote the set of all successor state-profile choices considered in the definition of T_i . Specifically, an element $\zeta \in \mathfrak{D}_i(\mathcal{C}, v_i)$ consists of the successor state(s) of v_i together with the corresponding cost profile(s) selected from $\mathcal{C}$. We further let

$$F_i : \mathfrak{D}_i(\mathcal{C}, v_i) \rightarrow \mathbb{R}_{\geq 0}^{\Theta_{\mathcal{K}}}$$

associate each such choice with the cost profile generated according to the defining expression of T_i . Accordingly, the value-iteration operator can be equivalently written as

$$\mathsf{T}_i(\mathcal{C}, v_i) = \text{Pareto}(\{F_i(\zeta) \mid \zeta \in \mathfrak{D}_i(\mathcal{C}, v_i)\}). \quad (28)$$

For any retained cost profile $\mathbf{c} \in \mathsf{T}_i(\mathcal{C}, v_i)$, we define $\arg_{\mathsf{T}_i}(\mathcal{C}, v_i, \mathbf{c})$ as an arbitrarily but fixedly selected successor state-profile choice satisfying

$$\arg_{\mathsf{T}_i}(\mathcal{C}, v_i, \mathbf{c}) \in \{\zeta \in \mathfrak{D}_i(\mathcal{C}, v_i) \mid F_i(\zeta) = \mathbf{c}\}. \quad (29)$$

Such a choice always exists since $\mathbf{c} \in \mathsf{T}_i(\mathcal{C}, v_i)$. Therefore $\arg_{\mathsf{T}_i}(\mathcal{C}, v_i, \mathbf{c})$ specifies one successor state-profile choice that generates the retained cost profile $\mathbf{c}$. To enable subsequent strategy reconstruction, we introduce a *generating-choice mapping* η that records, for each retained state-profile pair, one generating choice associated with its cost profile. At an X -state, the recorded choice determines the control successor, whereas at Y - and Z -states, it associates each possible successor branch with the corresponding cost profile.

With the above value-iteration operators and generating choices, we are ready to present the Pareto value-iteration procedure. The algorithm iteratively computes the Pareto-optimal cost-profile sets while storing the corresponding generating choices in the generating-choice mapping η , as summarized in Algorithm 2.

Algorithm 2: Pareto-Profile Value Iteration

Input: TTS $\mathcal{G}$ and the realization set Θ
Output: Pareto-optimal cost-profile sets $\{\mathcal{C}^*(v)\}_{v \in V}$ and the generating-choice mapping η

// Initialization

```

1  $\eta \leftarrow \emptyset$ ;
2  $\mathcal{C}^{(0)}(v) \leftarrow \emptyset, \forall v \in V$ ;
3 foreach  $v_x \in V_m$  do
4    $\mathbf{c}_{v_x}^{(0)}(\theta) \leftarrow 0, \forall \theta \in \Theta_{\mathcal{K}}$ ;
5    $\mathcal{C}^{(0)}(v_x) \leftarrow \{\mathbf{c}_{v_x}^{(0)}\}$ ;
6  $k \leftarrow 0$ ;
// Value Iteration
7 repeat
8    $k \leftarrow k + 1$ ;
9    $\mathcal{C}^{(k)} \leftarrow \mathcal{C}^{(k-1)}$ ;
10  foreach  $v_x \in V_X \setminus V_m$  do
11     $\mathcal{C}^{(k)}(v_x) \leftarrow \mathsf{T}_X(\mathcal{C}^{(k-1)}, v_x)$ ;
12    foreach  $\mathbf{c} \in \mathcal{C}^{(k)}(v_x)$  do
13      if  $(v_x, \mathbf{c}) \notin \text{dom}(\eta)$  then
14         $\eta(v_x, \mathbf{c}) \leftarrow \arg_{\mathsf{T}_X}(\mathcal{C}^{(k-1)}, v_x, \mathbf{c})$ ;
15  foreach  $v_y \in V_Y$  do
16     $\mathcal{C}^{(k)}(v_y) \leftarrow \mathsf{T}_Y(\mathcal{C}^{(k-1)}, v_y)$ ;
17    foreach  $\mathbf{c} \in \mathcal{C}^{(k)}(v_y)$  do
18      if  $(v_y, \mathbf{c}) \notin \text{dom}(\eta)$  then
19         $\eta(v_y, \mathbf{c}) \leftarrow \arg_{\mathsf{T}_Y}(\mathcal{C}^{(k-1)}, v_y, \mathbf{c})$ ;
20  foreach  $v_z \in V_Z \setminus \{d\}$  do
21     $\mathcal{C}^{(k)}(v_z) \leftarrow \mathsf{T}_Z(\mathcal{C}^{(k-1)}, v_z)$ ;
22    foreach  $\mathbf{c} \in \mathcal{C}^{(k)}(v_z)$  do
23      if  $(v_z, \mathbf{c}) \notin \text{dom}(\eta)$  then
24         $\eta(v_z, \mathbf{c}) \leftarrow \arg_{\mathsf{T}_Z}(\mathcal{C}^{(k-1)}, v_z, \mathbf{c})$ ;
25 until  $\mathcal{C}^{(k)}(v) = \mathcal{C}^{(k-1)}(v), \forall v \in V$ ;
26  $\mathcal{C}^* \leftarrow \mathcal{C}^{(k)}$ ;
27 return  $\{\mathcal{C}^*(v)\}_{v \in V}, \eta$ ;

```

The intuition of Algorithm 2 is explained as follows. The algorithm starts from zero cost profiles at the marked states and propagates the cost-profile sets backward through the

TTS by repeatedly applying the three value-iteration operators. Whenever a new retained state-profile pair is generated, one corresponding generating choice is stored in the generating-choice mapping η . For any retained state-profile pair $(v_x, \mathbf{c}_{v_x})$, an X -strategy can then be reconstructed recursively through η as follows. At an X -state, the successor Y -state recorded by η determines the control decision, whereas after the subsequent Y - and Z -transitions are resolved, the cost profile associated by η with the realized successor is followed recursively.

Example 6 (Running Example Cont.). *We continue with the TTS in Figure 2. Algorithm 2 initializes the marked states with zero cost profiles. In particular, $\mathcal{C}^{(0)}(x_5, \mathcal{K}_0) = \{(0, 0)\}$, $\mathcal{C}^{(0)}(x_5, \mathcal{K}_1) = \{(0)\}$, and $\mathcal{C}^{(0)}(x_5, \mathcal{K}_2) = \{(0)\}$. The converged Pareto-optimal cost-profile set $\mathcal{C}^*(v)$ at each state v is shown in blue next to the corresponding state in Figure 2.*

At $(x_1, \mathcal{K}_0)$, both realizations remain compatible and $\mathcal{C}^*(x_1, \mathcal{K}_0) = \{(12, 12)\}$. Thus, the retained profile incurs cost 12 under both θ_1 and θ_2 ; the profiles generated by the alternative control decisions are Pareto-dominated and are removed by Algorithm 2. Finally, at the initial state, we obtain $\mathcal{C}^*(v_0) = \{(22, 22), (5, 26)\}$, where the two components are ordered according to (θ_1, θ_2) . The profile $(22, 22)$ corresponds to a strategy that avoids exploring the unknown state x_3 and incurs cost 22 under either realization. In contrast, $(5, 26)$ corresponds to first exploring x_3 , resulting in cost 5 under θ_1 , where the shortcut is available, and cost 26 under θ_2 , where it is unavailable. Since neither profile Pareto-dominates the other, both are retained in $\mathcal{C}^*(v_0)$.

Before analyzing the optimality of the returned cost profiles, we first establish that Algorithm 2 reaches a fixed point after finitely many iterations.

Proposition 2. *For the finite TTS $\mathcal{G}$ constructed with non-negative transition costs, Algorithm 2 terminates after finitely many iterations.*

Proof. For each state $v \in V$ and each $k \geq 0$, let $\mathcal{A}_k(v)$ denote the set of all cost profiles that can be generated at v by recursively applying the three value-iteration operators, such that every play involved in the generation reaches a marked state within at most k transitions. We first claim that

$$\mathcal{C}^{(k)}(v) = \text{Pareto}(\mathcal{A}_k(v)), \quad \forall v \in V. \quad (30)$$

We prove the claim by induction on k . For $k = 0$, it follows directly from the initialization of Algorithm 2. Suppose that the claim holds for $k - 1$. The operators T_X , T_Y , and T_Z propagate the cost profiles at successor states backward by one TTS transition. Moreover, each operator preserves the componentwise order, that is, if a successor profile is replaced by another profile that is componentwise no larger, then the generated profile at the current state is also componentwise no larger. Therefore, removing dominated profiles at iteration $k - 1$ does not affect the Pareto frontier generated at iteration k . Since the three operators consider all successor state-profile choices specified in Definition 12, applying them to $\mathcal{C}^{(k-1)}$ yields exactly $\text{Pareto}(\mathcal{A}_k(v))$. Hence, (30) holds for k .

Now, consider a generating play that visits the same X -state $v_x = (x, \mathcal{K})$ twice, with cost profiles $\mathbf{c}$ and $\mathbf{c}'$ associated with

its first and second occurrences, respectively. Let ρ_c denote the cycle segment between them. By Proposition 1, no new knowledge is acquired along ρ_c . Hence, for every $\theta \in \Theta_K$,

$$\mathbf{c}(\theta) \geq \text{cost}_{\mathcal{G}}(\rho_c) + \mathbf{c}'(\theta).$$

Since all transition costs are nonnegative, $\text{cost}_{\mathcal{G}}(\rho_c) \geq 0$, and thus $\mathbf{c}' \preceq \mathbf{c}$. Therefore, the cycle can be removed without increasing any component of the cost profile.

Repeating the above cycle elimination yields a componentwise no-larger cost profile whose generating plays contain no repeated X -state. Since the TTS is finite and follows the structure $X \rightarrow Y \rightarrow Z \rightarrow X$, every such play reaches a marked state within at most $N = 3|V_X|$ transitions. Hence, for every $k \geq N$ and every $\mathbf{c} \in \mathcal{A}_k(v)$, there exists $\hat{\mathbf{c}} \in \mathcal{A}_N(v)$ such that $\hat{\mathbf{c}} \preceq \mathbf{c}$. Since $\mathcal{A}_N(v) \subseteq \mathcal{A}_k(v)$, we have

$$\text{Pareto}(\mathcal{A}_k(v)) = \text{Pareto}(\mathcal{A}_N(v)), \quad \forall k \geq N.$$

Therefore, by (30),

$$\mathcal{C}^{(k)}(v) = \mathcal{C}^{(N)}(v), \quad \forall v \in V, \quad \forall k \geq N,$$

which proves that Algorithm 2 terminates after finitely many iterations. The proof is thus completed. $\square$

We next show that every winning X -strategy can be matched by a strategy induced by a retained Pareto cost profile whose realization-wise worst-case cost is no larger.

Proposition 3. *Let $\{\mathcal{C}^*(v)\}_{v \in V}$ and η be returned by Algorithm 2. For any winning X -strategy $\pi_x \in \Pi_X$, there exists a cost profile $\mathbf{c} \in \mathcal{C}^*(v_0)$ and a winning X -strategy $\pi'_x \in \Pi_X$ reconstructed by recursively following η from $(v_0, \mathbf{c})$ such that*

- (i) $\mathbf{c}$ is the realization-wise worst-case cost profile induced by π'_x , i.e.,

$$\mathbf{c}(\theta) = \max_{\pi_y \in \Pi_Y} \text{cost}_{\mathcal{G}}(\rho_{\pi'_x, \pi_y, \pi_z^\theta}), \quad \forall \theta \in \Theta;$$

- (ii) π'_x incurs no larger worst-case cost than π_x under any realization, i.e.,

$$\mathbf{c}(\theta) \leq \max_{\pi_y \in \Pi_Y} \text{cost}_{\mathcal{G}}(\rho_{\pi_x, \pi_y, \pi_z^\theta}), \quad \forall \theta \in \Theta.$$

Proof. Given a winning X -strategy $\pi_x \in \Pi_X$, we define its realization-wise worst-case cost profile by

$$\bar{\mathbf{c}}(\theta) := \max_{\pi_y \in \Pi_Y} \text{cost}_{\mathcal{G}}(\rho_{\pi_x, \pi_y, \pi_z^\theta}), \quad \forall \theta \in \Theta. \quad (31)$$

Since π_x is winning, we know that, for any $\theta \in \Theta$ and any $\pi_y \in \Pi_Y$, we have $\text{last}(\rho_{\pi_x, \pi_y, \pi_z^\theta}) \in V_m$. Since $\pi_x \in \Pi_X$ and both the TTS and Θ are finite, there exists $h \in \mathbb{N}$ such that

$$|\rho_{\pi_x, \pi_y, \pi_z^\theta}| - 1 \leq h, \quad \forall \pi_y \in \Pi_Y, \quad \forall \theta \in \Theta.$$

Indeed, otherwise finite branching would yield an infinite outcome avoiding V_m , contradicting that π_x is winning.

By backward evaluation of these outcomes using T_X , T_Y , and T_Z , we have $\bar{\mathbf{c}} \in \mathcal{A}_h(v_0)$. Let N be the finite bound established in the proof of Proposition 2 and let $H = \max\{h, N\}$. Then we have $\bar{\mathbf{c}} \in \mathcal{A}_H(v_0)$ and $\mathcal{C}^*(v_0) = \text{Pareto}(\mathcal{A}_H(v_0))$. Since $\mathcal{A}_H(v_0)$ is finite, there exists $\mathbf{c} \in \mathcal{C}^*(v_0)$ such that

$$\mathbf{c} \preceq \bar{\mathbf{c}}. \quad (32)$$

We next consider the X -strategy reconstructed from $(v_0, \mathbf{c})$ through η . Suppose that a state-profile pair $(v, \mathbf{d})$ is first generated at iteration $k > 0$. By Algorithm 2, $\eta(v, \mathbf{d})$ is recorded using $\mathcal{C}^{(k-1)}$, and hence every successor profile in the recorded generating choice has been generated at an earlier iteration. Therefore, the generation iteration strictly decreases when recursively following η . Since only the zero profiles at marked states are initialized at iteration zero, every branch of the reconstruction reaches a marked state after finitely many transitions. Thus, the reconstructed strategy π'_x is winning.

It remains to show that $\mathbf{c}$ is exactly the realization-wise worst-case cost profile induced by π'_x . We prove by induction on the iteration at which a state-profile pair is first generated that

$$\mathbf{d}(\theta) = \max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi'_x, \pi_y, \pi_z}^v), \quad (33)$$

for every state-profile pair $(v, \mathbf{d})$ appearing in the reconstruction and every realization θ compatible with the knowledge at v , where $\rho_{\pi'_x, \pi_y, \pi_z}^v$ denotes the outcome play starting from v .

The claim holds for the pairs initialized at iteration zero, since they are precisely the zero profiles at marked states. Now consider a pair $(v, \mathbf{d})$ first generated at iteration $k > 0$. By Algorithm 2, all successor profiles recorded in $\eta(v, \mathbf{d})$ belong to $\mathcal{C}^{(k-1)}$, and hence the induction hypothesis applies to them.

If $v = v_x \in V_X \setminus V_m$ and $\eta(v_x, \mathbf{d}) = (v_y, \mathbf{d}_{v_y})$, then, by the definition of $\arg_{T_X}$ and the induction hypothesis, we have

$$\begin{aligned} \mathbf{d}(\theta) &= w_G(v_x, v_y) + \mathbf{d}_{v_y}(\theta) \\ &= w_G(v_x, v_y) + \max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi'_x, \pi_y, \pi_z}^{v_y}) \\ &= \max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi'_x, \pi_y, \pi_z}^{v_x}), \end{aligned}$$

where the last equality follows since π'_x selects v_y at v_x .

If $v = v_y \in V_Y$ and $\eta(v_y, \mathbf{d}) = \{(v_z, \mathbf{d}_{v_z})\}_{v_z \in \text{Succ}(v_y)}$, then, by the definition of $\arg_{T_Y}$ and the induction hypothesis, we have

$$\begin{aligned} \mathbf{d}(\theta) &= \max_{v_z \in \text{Succ}(v_y)} [w_G(v_y, v_z) + \mathbf{d}_{v_z}(\theta)] \\ &= \max_{v_z \in \text{Succ}(v_y)} \left[w_G(v_y, v_z) + \max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi'_x, \pi_y, \pi_z}^{v_z}) \right] \\ &= \max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi'_x, \pi_y, \pi_z}^{v_y}). \end{aligned}$$

The last equality follows since Π_Y contains all admissible Y -strategies, so any successor in $\text{Succ}(v_y)$ may be selected at the current history and followed by an arbitrary admissible Y -strategy thereafter.

Let $v = v_z = (x, \mathcal{K}, x') \in V_Z \setminus \{d\}$ and denote $v_x^\theta = f_{ZX}(v_z, \Lambda_{G_\theta}(x'))$. By the definition of $\arg_{T_Z}$ and the induction hypothesis, we have

$$\begin{aligned} \mathbf{d}(\theta) &= w_G(v_z, v_x^\theta) + \mathbf{d}_{v_x^\theta}(\theta) \\ &= w_G(v_z, v_x^\theta) + \max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi'_x, \pi_y, \pi_z}^{v_x^\theta}) \\ &= \max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi'_x, \pi_y, \pi_z}^{v_z}), \end{aligned}$$

where the last equality follows since π_z^θ uniquely selects v_x^θ at v_z . Therefore, (33) holds for all state-profile pairs in the reconstruction. In particular, at $(v_0, \mathbf{c})$, we know that (i) holds.

Algorithm 3: Regret-Optimal Strategy Synthesis

Input: TTS $\mathcal{G}$, Pareto-optimal cost-profile sets $\mathcal{C}^*$, generating-choice mapping η , and realization-wise best responses $\{\text{BR}(\theta)\}_{\theta \in \Theta}$

Output: A regret-optimal strategy π_x^* and the optimal regret value Reg^*

// Regret-Optimal Profile Selection

1 **if** $\mathcal{C}^*(v_0) = \emptyset$ **then**

2 **return** *No winning strategy exists*;

3 $\mathbf{c}^* \leftarrow \text{select} \left(\underset{\mathbf{c} \in \mathcal{C}^*(v_0)}{\text{argmin}} \max_{\theta \in \Theta_{K_0}} \{\mathbf{c}(\theta) - \text{BR}(\theta)\} \right);$

4 $\text{Reg}^* \leftarrow \max_{\theta \in \Theta_{K_0}} \{\mathbf{c}^*(\theta) - \text{BR}(\theta)\};$

// Strategy Reconstruction

5 $\mathcal{W} \leftarrow \{(v_0, \mathbf{c}^*)\};$

6 $\pi_x^* \leftarrow \emptyset;$

7 **while** $\mathcal{W} \neq \emptyset$ **do**

8 $(v_x, \mathbf{c}_{v_x}) \leftarrow \text{select}(\mathcal{W});$

9 $\mathcal{W} \leftarrow \mathcal{W} \setminus \{(v_x, \mathbf{c}_{v_x})\};$

10 **if** $v_x \in V_m$ **then**

11 $\pi_x^*(v_x, \mathbf{c}_{v_x}) \leftarrow \text{Null};$

12 **else**

13 $(v_y, \mathbf{c}_{v_y}) \leftarrow \eta(v_x, \mathbf{c}_{v_x});$

14 $\pi_x^*(v_x, \mathbf{c}_{v_x}) \leftarrow v_y;$

15 $\{(v_z, \mathbf{c}_{v_z})\}_{v_z \in \text{Succ}(v_y)} \leftarrow \eta(v_y, \mathbf{c}_{v_y});$

16 **foreach** $v_z \in \text{Succ}(v_y)$ **do**

17 $\{(v'_x, \mathbf{c}_{v'_x})\}_{v'_x \in \text{Succ}(v_z)} \leftarrow \eta(v_z, \mathbf{c}_{v_z});$

18 **foreach** $v'_x \in \text{Succ}(v_z)$ **do**

19 **if** $(v'_x, \mathbf{c}_{v'_x}) \notin \text{dom}(\pi_x^*)$ **then**

20 $\mathcal{W} \leftarrow \mathcal{W} \cup \{(v'_x, \mathbf{c}_{v'_x})\};$

21 **return** $\pi_x^*, \text{Reg}^*;$

Finally, by (32) and (31), we know that (ii) holds. The proof is thus completed. $\square$

With the Pareto-optimal cost-profile sets $\{\mathcal{C}^*(v)\}_{v \in V}$ and the generating-choice mapping η returned by Algorithm 2, we can now synthesize a regret-optimal X -strategy, summarized in Algorithm 3. The intuition is explained as follows. The intuition is explained as follows. The algorithm first checks whether the root admits any retained cost profile. If so, it selects from $\mathcal{C}^*(v_0)$ a profile minimizing the worst-case difference between the profile cost and the realization-wise best response. The corresponding winning X -strategy is then reconstructed through the generating-choice mapping η .

Before establishing the correctness of Algorithm 3, we first revisit the running example to illustrate how the retained cost profiles are used for regret evaluation and how the corresponding regret-optimal X -strategy is reconstructed through η .

Example 7 (Running Example Cont.). Recall from Example 5 that the two components of a cost profile at the initial knowledge set K_0 are ordered according to (θ_1, θ_2) , where θ_1 corresponds to the realization in which the shortcut is available and θ_2 to the realization in which it is unavailable.

From the fully-known min-max problems, we have $BR(\theta_1) = 5$ and $BR(\theta_2) = 22$.

At the initial state, Algorithm 2 returns $C^*(v_0) = \{(22, 22), (5, 26)\}$. For the profile $(22, 22)$, the corresponding regret is $\max\{22 - 5, 22 - 22\} = 17$, whereas for $(5, 26)$ it is $\max\{5 - 5, 26 - 22\} = 4$. Hence, Algorithm 3 selects $c^* = (5, 26)$ and returns $Reg^* = 4$. The generating-choice mapping η associated with c^* is then used to reconstruct the corresponding X -strategy. The resulting strategy is highlighted by the blue arrows in Figure 2.

Remark 3. By Proposition 2, Algorithm 2 terminates within $3|V_X| \leq 3|V|$ iterations. In each iteration, all states and transitions of the TTS are processed, while the additional computation arises from combining and Pareto-pruning the realization-wise cost profiles. Hence, apart from the Pareto-profile operations, the computation is polynomial in the TTS size $|V| + |E|$. Algorithm 3 subsequently performs only a scan of the profiles retained at the initial state and reconstructs the strategy through η . Together with Remark 2, the dominant combinatorial growth is associated with the number of unknown states and the resulting structural uncertainty. In many practical systems, however, unknown states constitute only a small fraction of the overall state space. Hence, for a fixed number of unknown states and a bounded number of distinct realization-wise cost outcomes induced at each TTS state, the proposed synthesis has polynomial complexity with respect to the size of the underlying DES.

Remark 4. The Pareto value iteration developed here is fundamentally different from the scalar value-iteration approach in our previous work [39]. In the partially-known planning setting considered there, once the environment realization is fixed, an agent strategy induces a deterministic outcome. This structure allows every relevant outcome to be represented by a shortest path between knowledge updates, so that realization-dependent regret can be encoded by a scalar terminal weight and solved through a standard min-max value iteration. This reduction no longer applies in the present setting. Even for a fixed realization, uncontrollable events may generate multiple possible outcomes under the same supervisory decision. Hence, the cost associated with each realization must itself account for the worst-case event occurrence, and different supervisory decisions may induce incomparable realization-wise cost tradeoffs. The proposed Pareto value iteration therefore retains these costs separately until the final regret evaluation. When $\Sigma_{uc} = \emptyset$, the event-occurrence player becomes trivial and the TTS reduces to the two-player partially-known planning setting of [39]. The proposed Pareto value iteration remains applicable and yields the same regret-optimal synthesis problem, whereas the previous scalar value iteration can be viewed as a specialized reduction that further exploits the deterministic structure of this special case. Thus, the present framework generalizes the previous setting while extending regret-optimal synthesis to systems with both structural uncertainty and nondeterministic event occurrences.

B. Correctness of the Algorithm

We now formally establish the correctness of the proposed synthesis algorithm. Specifically, we show that Algorithm 3 is

- (i) **sound:** if Algorithm 3 returns an X -strategy π_x^* , then π_x^* is winning and $Reg_G(\pi_x^*) = Reg^*$;
- (ii) **complete:** if $C^*(v_0) = \emptyset$, then no winning X -strategy exists, i.e., $\Pi_X = \emptyset$;
- (iii) **optimal:** for any winning X -strategy $\pi_x \in \Pi_X$, we have

$$Reg_G(\pi_x^*) \leq Reg_G(\pi_x).$$

First, we show the soundness of our algorithm. Before this, we need to establish that every cost profile returned by the Pareto value-iteration procedure indeed corresponds to a realizable winning X -strategy.

Proposition 4. Let $\{C^*(v)\}_{v \in V}$ and η be returned by Algorithm 2. For any $c \in C^*(v_0)$, let π_x^c denote the X -strategy reconstructed from (v_0, c) through η . Then π_x^c is a winning X -strategy. Moreover, for every realization $\theta \in \Theta$,

$$c(\theta) = \max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi_x^c, \pi_y, \pi_z^\theta}). \quad (34)$$

Proof. Let $c \in C^*(v_0)$ be arbitrary. Recall the reconstruction argument in the proof of Proposition 3. We note that this argument only relies on c being a retained cost profile and the construction of the generating-choice mapping η . Specifically, the generation iteration strictly decreases along η and therefore eventually reaches an iteration-zero profile at a marked state. Hence, π_x^c is winning.

Moreover, the induction argument in the proof of Proposition 3 shows that, for every state-profile pair (v, d) encountered in the reconstruction and every realization θ compatible with the knowledge at v , we have

$$d(\theta) = \max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi_x^c, \pi_y, \pi_z^\theta}^v).$$

Applying this equality to (v_0, c) yields (34). $\square$

Now, we present the soundness of our algorithm as follows.

Proposition 5. If Algorithm 3 returns (π_x^*, Reg^*) , then π_x^* is winning and

$$Reg_G(\pi_x^*) = Reg^*.$$

Proof. Since Algorithm 3 returns (π_x^*, Reg^*) , we have $c^* \in C^*(v_0)$. By Proposition 4, π_x^* is winning and

$$c^*(\theta) = \max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi_x^*, \pi_y, \pi_z^\theta}), \quad \forall \theta \in \Theta. \quad (35)$$

Recall that every realization $\theta \in \Theta$ uniquely induces the compatible DES G_θ and the strongly positional Z -strategy π_z^θ . For a fixed realization θ , fixing the Z -strategy to π_z^θ restricts the TTS to the transition structure of G_θ , while the Y -player represents all possible occurrences of enabled events. Therefore, the minimum worst-case cost achievable when θ is known a priori is exactly

$$BR(\theta) = \min_{S \in \Psi_W(G_\theta)} \text{cost}(S/G_\theta).$$

Hence, the realization-wise regret of π_x^* under θ is

$$\max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi_x^*, \pi_y, \pi_z^\theta}) - BR(\theta).$$

Since $\Pi_Z = \{\pi_z^\theta : \theta \in \Theta\}$, taking the worst case over all realizations gives

$$\begin{aligned} \text{Reg}_G(\pi_x^*) &= \max_{\theta \in \Theta} \left[\max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi_x^*, \pi_y, \pi_z^\theta}) - \text{BR}(\theta) \right] \\ &= \max_{\theta \in \Theta} [\mathbf{c}^*(\theta) - \text{BR}(\theta)] \\ &= \text{Reg}^*, \end{aligned}$$

where the second equality follows from (35), and the last equality directly follows from Algorithm 3. The proof is thus completed. $\square$

Now, we present the completeness of our algorithm.

Proposition 6. *If $\mathcal{C}^*(v_0) = \emptyset$, then no winning X -strategy exists, i.e., $\Pi_X = \emptyset$.*

Proof. We prove the result by contraposition. Suppose that there exists a winning X -strategy $\pi_x \in \Pi_X$. By Proposition 3, there exists a cost profile $\mathbf{c} \in \mathcal{C}^*(v_0)$. Hence, $\mathcal{C}^*(v_0) \neq \emptyset$. Therefore, $\mathcal{C}^*(v_0) = \emptyset$ implies $\Pi_X = \emptyset$. $\square$

Finally, we establish the optimality of our algorithm, which completes the correctness proof.

Theorem 1. *Let (π_x^*, Reg^*) be returned by Algorithm 3. Then π_x^* is a regret-optimal winning X -strategy and Reg^* is its regret value. Formally,*

$$\text{Reg}^* = \text{Reg}_G(\pi_x^*) \leq \text{Reg}_G(\pi_x), \quad \forall \pi_x \in \Pi_X. \quad (36)$$

Proof. By Proposition 5, π_x^* is winning and $\text{Reg}_G(\pi_x^*) = \text{Reg}^*$. Now, consider an arbitrary winning X -strategy $\pi_x \in \Pi_X$ and define its realization-wise worst-case cost profile as

$$\bar{\mathbf{c}}(\theta) = \max_{\pi_y \in \Pi_Y} \text{cost}_G(\rho_{\pi_x, \pi_y, \pi_z^\theta}), \quad \forall \theta \in \Theta.$$

By Proposition 3, there exists $\mathbf{c} \in \mathcal{C}^*(v_0)$ such that

$$\mathbf{c}(\theta) \leq \bar{\mathbf{c}}(\theta), \quad \forall \theta \in \Theta.$$

Therefore, we have

$$\max_{\theta \in \Theta} [\mathbf{c}(\theta) - \text{BR}(\theta)] \leq \max_{\theta \in \Theta} [\bar{\mathbf{c}}(\theta) - \text{BR}(\theta)] = \text{Reg}_G(\pi_x). \quad (37)$$

Since Algorithm 3 selects $\mathbf{c}^*$ from $\mathcal{C}^*(v_0)$ to minimize the worst-case regret, we have

$$\text{Reg}^* = \max_{\theta \in \Theta} [\mathbf{c}^*(\theta) - \text{BR}(\theta)] \leq \max_{\theta \in \Theta} [\mathbf{c}(\theta) - \text{BR}(\theta)].$$

Together with (37), we obtain

$$\text{Reg}_G(\pi_x^*) = \text{Reg}^* \leq \text{Reg}_G(\pi_x).$$

The proof is thus completed. $\square$

VI. CASE STUDIES

In this section, we present a case study to illustrate the effectiveness of the proposed regret-optimal supervisory control approach in a partially-known environment. Specifically, we consider a mobile robot with a reachability objective in a dynamic workspace. All simulations are conducted in V-REP, and the corresponding video is available at

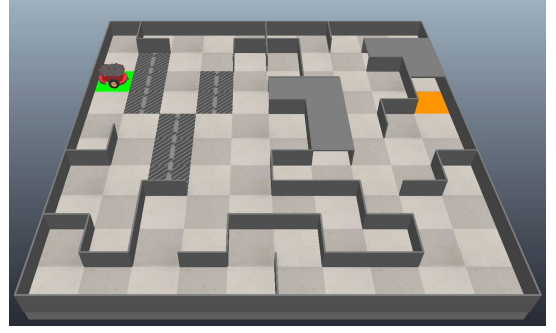


Fig. 3. Scenario for robot path planning in

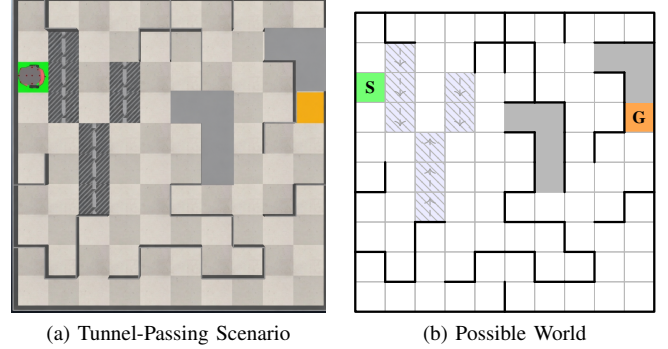


Fig. 4. Tunnel-Passing scenario and its corresponding possible world.

The workspace contains three conveyors and roof-covered regions with unknown transition structures, as illustrated in Figure 3. At each step, the robot can choose left, right, up, or down, provided that the resulting motion does not collide with the walls. Once the robot enters a conveyor, however, the resulting conveyor-induced motion is not under its control and is therefore modeled by uncontrollable events. In addition, the transition structures under the roof-covered regions are fixed in each realization but initially unknown. Distinct event patterns and unknown states are introduced to represent such uncertainties. Therefore, the robot motion can be modeled as a PK-DES, with the objective of reaching the target in every compatible realization while minimizing the trajectory cost. Using Algorithms 2 and 3, we synthesize the regret-optimal strategy. The resulting strategy explores the unknown region in the middle but not the one on the right. If the shortcut in the middle is available, exploration saves 10 units of cost compared with avoiding the region; if it is unavailable, the additional exploration cost is at most 8 units. In contrast, exploring the region on the right does not provide sufficient potential cost reduction. Accordingly, the minimized regret is $\text{Reg}^* = 8$.

We consider four compatible realizations G_1 – G_4 , which differ in the conveyor status and the connectivity under the middle roof-covered region, as shown in Figure 5. Under the regret-optimal strategy, the robot moves toward the middle unknown region and explores it after passing the conveyors. The trajectories differ because the strategy reacts online to the uncontrollable conveyor-induced motions. In G_1 and G_2 , the shortcut is available, so the robot directly uses it to

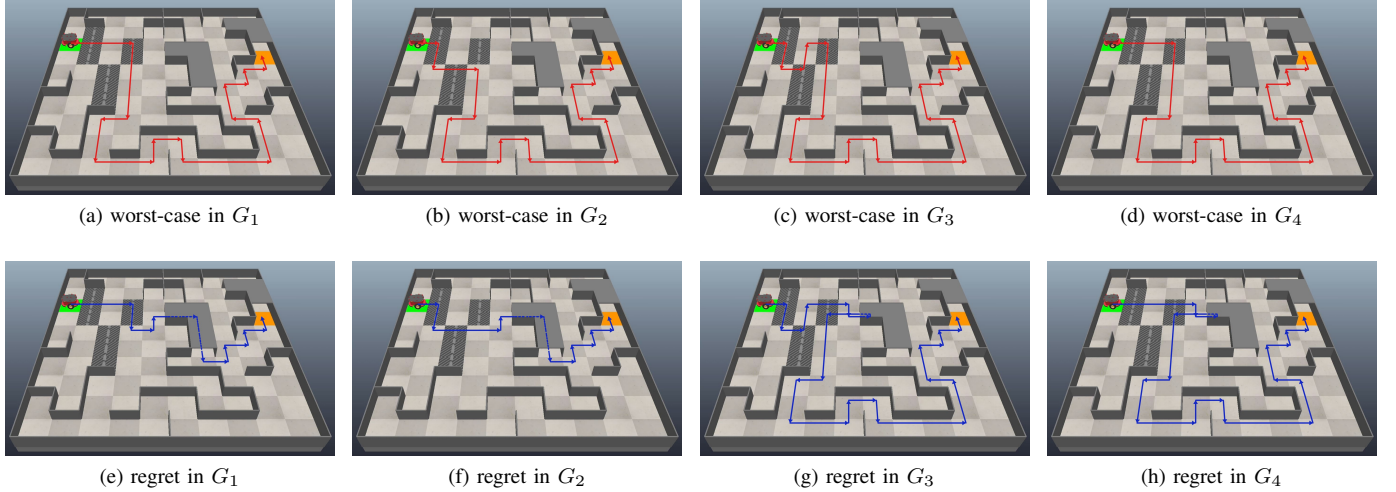


Fig. 5. Trajectories of the robot under the worst-case-based strategy (red) and the regret-optimal strategy (blue) in four compatible realizations G_1 – G_4 . The middle region is passable in G_1 and G_2 and blocked in G_3 and G_4 . Dashed segments indicate the part of the invisible trajectory under the roof.

reach “G”. In G_3 and G_4 , the shortcut is unavailable, so the robot backtracks and follows a longer route, incurring only 4 additional units of exploration cost. For comparison, the worst-case-based strategy avoids exploration and behaves as if the shortcut were always unavailable. Consequently, it always follows the longer route around the unknown region, as shown in Figures 5a–5d. Compared with the regret-optimal strategy, it incurs 10 additional units of cost in G_1 and G_2 , while saving only 4 units in G_3 and G_4 .

Overall, these results show that the regret-optimal strategy reduces the conservatism of worst-case planning by exploiting beneficial unknown transitions while keeping the cost of unsuccessful exploration limited.

VII. CONCLUSION

In this paper, we formulated and solved the regret-optimal supervisory control problem for partially-known discrete-event systems. The proposed PK-DES framework captures structural information that is fixed in the actual plant but initially unknown and can be progressively revealed through exploration. By incorporating the evolution of such knowledge with supervisory decisions and uncontrollable event occurrences, we constructed a tripartite transition system and formulated the problem as a three-player graph game. We then developed an exact Pareto value-iteration algorithm that computes the relevant realization-wise cost tradeoffs without explicitly enumerating supervisory strategies. Finite termination, soundness, completeness, and regret optimality of the resulting synthesis procedure were established. The robotic case study further demonstrated that regret provides an effective means of balancing the potential benefit of exploration against its possible cost under partially-known transition structures.

Future work includes extending this framework to more general specifications such as cyclic tasks and long-run objectives, as well as investigating partially-known systems under additional forms of information uncertainty.

REFERENCES

- [1] Lucas VR Alves, Patrícia N Pena, and Ricardo HC Takahashi. Planning on discrete event systems using parallelism maximization. *Control Engineering Practice*, 112:104813, 2021.
- [2] Thomas Brihaye, Gilles Geeraerts, Axel Haddad, and Benjamin Monmege. Pseudopolynomial iterative algorithm to solve total-payoff games and min-cost reachability games. *Acta Informatica*, 54(1):85–125, 2017.
- [3] Michaël Cadilhac, Guillermo A Pérez, and Marie van den Bogaard. The impatient may use limited optimism to minimize regret. In *International Conference on Foundations of Software Science and Computation Structures*, pages 133–149. Springer, 2019.
- [4] C. Cassandras and S. Lafortune. *Introduction to Discrete Event Systems*. Springer, 2008.
- [5] Krishnendu Chatterjee, Mickael Randour, and Jean-François Raskin. Strategy synthesis for multi-dimensional quantitative objectives. *Acta informatica*, 51(3):129–163, 2014.
- [6] Randy Cieslak, C Desclaux, Ayman S Fawaz, and Pravin Varaiya. Supervisory control of discrete-event processes with partial observations. *IEEE transactions on automatic control*, 33(3):249–260, 1988.
- [7] Emmanuel Filiot, Tristan Le Gall, and Jean-François Raskin. Iterated regret minimization in game graphs. In *International Symposium on Mathematical Foundations of Computer Science*, pages 342–354, 2010.
- [8] Jinbo Fu, Asok Ray, and Constantino M Lagoa. Unconstrained optimal control of regular languages. *Automatica*, 40(4):639–646, 2004.
- [9] Lenko Grigorov and Karen Rudie. Near-optimal online control of dynamic discrete-event systems. *Discrete Event Dynamic Systems*, 16:419–449, 2006.
- [10] Michael Heymann and Feng Lin. Discrete-event control of nondeterministic systems. *IEEE Transactions on Automatic Control*, 43(1):3–17, 1998.
- [11] Paul Hunter, Guillermo A Pérez, and Jean-François Raskin. Minimizing regret in discounted-sum games. *arXiv preprint arXiv:1511.00523*, 2015.
- [12] Paul Hunter, Guillermo A Pérez, and Jean-François Raskin. Reactive synthesis without regret. *Acta Informatica*, 54(1):3–39, 2017.
- [13] Yiding Ji, Xiang Yin, and Stéphane Lafortune. Optimal supervisory control with mean payoff objectives and under partial observation. *Automatica*, 123:109359, 2021.
- [14] Jan Komenda and Tomáš Masopust. Supervisory control of modular discrete-event systems under partial observation: Normality. *IEEE Transactions on Automatic Control*, 69(6):3796–3807, 2023.
- [15] Naoki Kushi and Shigemasa Takai. Synthesis of similarity enforcing supervisors for nondeterministic discrete event systems. *IEEE Transactions on Automatic Control*, 63(5):1457–1464, 2017.
- [16] Jinglun Li and Shigemasa Takai. Maximally permissive supervisors for nonblocking similarity control of nondeterministic discrete-event systems. *IEEE Transactions on Automatic Control*, 68(6):3529–3544, 2022.

- [17] Jinglun Li and Shigemasa Takai. Synthesis of maximally permissive supervisors for similarity control of partially observed nondeterministic discrete event systems. *Automatica*, 135:109978, 2022.
- [18] Xinkai Liang, Huixuan Chan, Ying Liu, Yangxi Shi, and Hao Fang. Min-max regret task allocation and planning of heterogeneous multi-robot system in partially known environments. *arXiv preprint arXiv:2607.13403*, 2026.
- [19] Peng Lv, Zhangcong Xu, Yiding Ji, Shaoyuan Li, and Xiang Yin. Optimal supervisory control of discrete event systems for cyclic tasks. *Automatica*, 164:111634, 2024.
- [20] Ziyue Ma and Kai Cai. Optimal secret protections in discrete-event systems. *IEEE Transactions on Automatic Control*, 67(6):2816–2828, 2021.
- [21] Hervé Marchand, Olivier Boivineau, and Stéphane Lafortune. On optimal control of a class of partially observed discrete event systems. *Automatica*, 38(11):1935–1943, 2002.
- [22] Rômulo Meira-Góes, Jack Weitz, and Stéphane Lafortune. A compact and uniform approach for synthesizing state-based property-enforcing supervisors for discrete-event systems. *IEEE Transactions on Automatic Control*, 67(7):3567–3573, 2021.
- [23] Karan Muvvala, Peter Amorese, and Morteza Lahijanian. Let’s collaborate: Regret-based reactive synthesis for robotic manipulation. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 4340–4346. IEEE, 2022.
- [24] Karan Muvvala and Morteza Lahijanian. Efficient symbolic approaches for quantitative reactive synthesis with finite tasks. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8666–8672. IEEE, 2023.
- [25] Sasinee Pruekprasert, Toshimitsu Ushio, and Takafumi Kanazawa. Quantitative supervisory control game for discrete event systems. *IEEE Transactions on Automatic Control*, 61(10):2987–3000, 2015.
- [26] P. Ramadge and W. Wonham. Supervisory control of a class of discrete event processes. *SIAM Journal on Control and Optimization*, 25(1):206–230, 1987.
- [27] Ning Ran, Tingting Li, Shouguang Wang, and Zhou He. Supervisor synthesis for petri nets with uncontrollable and unobservable transitions. *IEEE Transactions on Automation Science and Engineering*, 21(2):1517–1525, 2023.
- [28] Klaus Werner Schmidt and Christian Breindl. A framework for state attraction of discrete event systems under partial observation. *Information Sciences*, 281:265–280, 2014.
- [29] Carla Seatzu, Manuel Silva, and Jan H Van Schuppen. *Control of discrete-event systems*, volume 433. Springer, 2013.
- [30] Raja Sengupta and Stéphane Lafortune. An optimal control theory for discrete event systems. *SIAM Journal on control and Optimization*, 36(2):488–541, 1998.
- [31] Shigemasa Takai. Bisimilarity enforcing supervisory control of nondeterministic discrete event systems with nondeterministic specifications. *Automatica*, 108:108470, 2019.
- [32] Shigemasa Takai. Synthesis of maximally permissive supervisors for nondeterministic discrete event systems with nondeterministic specifications. *IEEE Transactions on Automatic Control*, 66(7):3197–3204, 2020.
- [33] Shigemasa Takai. Polynomial-time verification for bisimilarity control of partially observed nondeterministic discrete event systems with deterministic specifications. *Automatica*, 152:110940, 2023.
- [34] Xiaojun Wang, Shaolong Shu, and Feng Lin. Supervisory control of networked discrete event systems with observation losses. *IEEE Transactions on Automatic Control*, 70(11):7303–7317, 2025.
- [35] W. Wonham and K. Cai. Supervisory control of discrete-event systems, 2019.
- [36] Walter Murry Wonham, Kai Cai, and Karen Rudie. Supervisory control of discrete-event systems: A brief history. *Annual Reviews in Control*, 45:250–256, 2018.
- [37] Xiang Yin and Stéphane Lafortune. Synthesis of maximally permissive supervisors for partially-observed discrete-event systems. *IEEE Transactions on Automatic Control*, 61(5):1239–1254, 2015.
- [38] Jianing Zhao, Bohan Cui, Dimos V Dimarogonas, Rupak Majumdar, and Xiang Yin. Regret-optimal supervisory control of partially-known discrete-event systems. In *2025 IEEE 64th Conference on Decision and Control (CDC)*, pages 1214–1219. IEEE, 2025.
- [39] Jianing Zhao, Keyi Zhu, Mingyang Feng, Shaoyuan Li, and Xiang Yin. No-regret path planning for temporal logic tasks in partially-known environments. *The International J. Robotics Research*, page 02783649251315758, 2025.